

UNIVERSIDADE FEDERAL DE ALAGOAS — UFAL
Campus A. C. Simões
Ciência da Computação

Hélio José Ribeiro Rêgo
Antônio Guilherme da Silva
Gabriel Nathan Telaroli Dias Calixto

Relatório Técnico: Desenvolvimento do Compilador para a Linguagem MiniPar
v2025.1

Maceió — AL
2025

1 Introdução.....	3
1.1 Objetivo.....	3
1.2 Escopo.....	3
1.2.1 Análise Léxica.....	3
1.2.2 Análise Sintática com Construção de Árvore Sintática Abstrata (AST).....	3
1.2.3 Análise Semântica.....	3
1.2.4 Tabela de Símbolos.....	3
1.2.5 Geração de Código Intermediário (TAC - Three-Address Code).....	4
1.2.6 Geração de Código Assembly (ARMv7).....	4
1.2.7 Executor em Tempo de Execução (Runtime Executor).....	4
2 A Linguagem MiniPar 2025.1.....	4
Gramática da Linguagem Minipar.....	4
Estrutura Geral.....	4
Precedência de Operadores (do menor para o maior).....	7
Elementos Léxicos.....	8
Palavras-chave.....	8
Tipos.....	8
Literais.....	9
Operadores.....	9
Delimitadores.....	9
3 Metodologia e Modelagem de Software.....	9
3.1 Gerenciamento de Projeto (Backlogs).....	9
3.1.1 Product Backlog.....	9
3.1.2 Sprint Backlog.....	10
3.2 Modelagem UML.....	11
3.2.1 Diagrama de Casos de Uso.....	12
3.2.2 Arquitetura de Componentes.....	12
3.2.3 Diagramas de Classe por Componente.....	13
4 Arquitetura e Implementação (Pseudocódigos).....	16
4.1 Analisador Léxico (Lexer).....	16
4.2 Analisador Sintático (Parser) e AST.....	19
4.2.1 AST.....	19
4.2.2 Analisador Sintático.....	20
4.3 Analisador Semântico e Tratamento de Erros.....	24
4.4 Tabela de Símbolos.....	32
4.5 Gerador de Código Intermediário (Três Endereços).....	34
4.6 Gerador de Código Assembly (ARMv7).....	39

5 Tecnologias Utilizadas.....	46
6 Testes e Validação.....	47
6.1 Interface do Compilador.....	47
6.2 Testes de Integração.....	48
6.3 Execução dos Programas de Teste.....	49
6.3.1 Programa de Teste 1: Calculadora Cliente/Servidor.....	49
6.3.2 Programa de Teste 2: Fatorial Iterativo (Geração de Assembly ARMv7 e execução no CPULator).....	49
6.3.3 Programa de Teste 3: Série de Fibonacci (Geração de Assembly ARMv7 e execução no CPULator).....	53
6.3.4 Programa de Teste 8: Quicksort Iterativo (Geração de Código Intermediário e Assembly ARMv7, com execução no CPULator).....	57
7 Conclusão.....	60
8 Anexos.....	61
8.1 Link para o Repositório (GitHub).....	61
8.2 Link para o Vídeo de apresentação do projeto (Youtube).....	61

1 Introdução

1.1 Objetivo

O presente projeto tem como objetivo principal o **desenvolvimento de um compilador completo para a linguagem de programação MiniPar, versão 2025.1**. Este compilador implementa uma pipeline de processamento de código-fonte estruturada em múltiplas fases especializadas, permitindo a tradução de programas escritos em MiniPar para código executável nativo.

O compilador proposto não se limita apenas à geração de código intermediário, mas constitui um sistema end-to-end capaz de processar um programa em MiniPar e produzi-lo em diferentes formatos de saída, desde representações intermediárias até código de máquina otimizado para plataformas específicas.

1.2 Escopo

O escopo do projeto abrange a implementação das seguintes fases do compilador, conforme os requisitos de engenharia de software moderna:

1.2.1 Análise Léxica

- **Responsabilidade:** Tokenização do código-fonte
- **Entrada:** Fluxo de caracteres do programa MiniPar
- **Saída:** Sequência de tokens (lexemas categorizados)
- **Implementação:** Módulo `lexer.py` (análise linear do código)

1.2.2 Análise Sintática com Construção de Árvore Sintática Abstrata (AST)

- **Responsabilidade:** Validação sintática e construção da AST
- **Entrada:** Sequência de tokens do lexer
- **Saída:** Árvore Sintática Abstrata (Abstract Syntax Tree)
- **Implementação:** Módulos `parser.py` e `ast_nodes.py`

1.2.3 Análise Semântica

- **Responsabilidade:** Validação de semântica, verificação de tipos e escopo
- **Entrada:** Árvore Sintática Abstrata
- **Saída:** AST validada e anotada com informações de tipo
- **Implementação:** Módulo `semantic.py`

1.2.4 Tabela de Símbolos

- **Responsabilidade:** Manutenção de registro de identificadores, tipos e escopos
- **Entrada:** Informações semânticas durante análise

- **Saída:** Estrutura de dados indexada de símbolos
- **Implementação:** Módulo `symbol_table.py`

1.2.5 Geração de Código Intermediário (TAC - Three-Address Code)

- **Responsabilidade:** Tradução da AST para representação intermediária
- **Entrada:** AST validada pela análise semântica
- **Saída:** Sequência de instruções de três endereços
- **Implementação:** Módulo `codegen.py`

1.2.6 Geração de Código Assembly (ARMv7)

- **Responsabilidade:** Tradução de código intermediário para instrução nativa
- **Entrada:** Código de três endereços
- **Saída:** Código Assembly ARMv7 compatível com processadores ARM
- **Implementação:** Módulo `c_codegen.py` (com suporte a backend)

1.2.7 Executor em Tempo de Execução (Runtime Executor)

- **Responsabilidade:** Execução direta de programas MiniPar
- **Entrada:** AST interpretável ou código intermediário
- **Saída:** Resultado da execução (saída padrão, valores retornados)
- **Implementação:** Módulo `runner.py`

2 A Linguagem MiniPar 2025.1

Gramática da Linguagem Minipar

Estrutura Geral

program → declarations

declarations → declaration declarations
| ε

declaration → var_decl
| func_decl
| channel_decl

var_decl → "var" type IDENTIFIER ("=" expression)? ";"

type → "number"
| "string"
| "bool"
| "void"
| "list"
| "dict"
| "any"

func_decl → "func" type IDENTIFIER "(" parameters? ")" block

parameters → var_decl ("," var_decl)*

block → "{" stmts "}"

stmts → stmt stmts
| ε

stmt → compound_stmt
| simple_stmt

simple_stmt → var_decl
| "return" expression? ";"
| "break" ";"
| "continue" ";"
| expr_stmt ";"

expr_stmt → assignment
| function_call

compound_stmt → if_stmt
| while_stmt
| seq_block
| par_block
| channel_stmt

if_stmt → "if" "(" expression ")" block ("else" block)?

while_stmt → "while" "(" expression ")" block

seq_block → "seq" block

par_block → "par" block

channel_stmt → s_channel_decl
| c_channel_decl

s_channel_decl → "s_channel" IDENTIFIER "(" channel_args ")" ";"

c_channel_decl → "c_channel" IDENTIFIER "(" channel_args ")" ";"

channel_args → argument ("," argument)*

argument → expression

assignment → IDENTIFIER "=" expression

function_call → IDENTIFIER "(" arguments? ")"

arguments → expression ("," expression)*

method_call → IDENTIFIER "." IDENTIFIER "(" arguments? ")"

expression → or_expr

or_expr → and_expr ("||" and_expr)*

and_expr → comparison_expr ("&&" comparison_expr)*

comparison_expr → additive_expr (comparison_op additive_expr)*

comparison_op → "=="

| "!="

| "<"

| ">"

| "<="

| ">="

additive_expr → multiplicative_expr (additive_op multiplicative_expr)*

additive_op → "+"

| "-"

multiplicative_expr → unary_expr (multiplicative_op unary_expr)*

multiplicative_op → "*"

| "/"

| "%"

unary_expr → unary_op unary_expr

| postfix_expr

unary_op → "!"

| "-"

postfix_expr → primary_expr

| method_call

| function_call

| postfix_expr "[" expression "]"

primary_expr → NUMBER_LITERAL

| STRING_LITERAL

| "true"

| "false"

| IDENTIFIER

| "(" expression ")"

| list_literal

| dict_literal

list_literal → "[" (expression ("," expression)*)? "]"

dict_literal → "{" (dict_pair ("," dict_pair)*)? "}"

dict_pair → expression ":" expression

Precedência de Operadores (do menor para o maior)

Precedência	Operador	Associatividade
1 (menor)	<code> </code> (OR lógico)	Esquerda
2	<code>&&</code> (AND lógico)	Esquerda
3	<code>==, !=, <, >, <=, >=</code> (Comparação)	Esquerda

4	<code>+</code> , <code>-</code> (Adição, Subtração)	Esquerda
5	<code>*</code> , <code>/</code> , <code>%</code> (Multiplicação, Divisão, Módulo)	Esquerda
6	<code>!</code> , <code>-</code> (NOT, Negação unária)	Direita
7 (maior)	<code>.</code> , <code>[]</code> , <code>()</code> (Acesso, Indexação, Chamada)	Esquerda

Elementos Léxicos

Palavras-chave

- `break`
- `continue`
- `else`
- `false`
- `func`
- `if`
- `par` (bloco paralelo)
- `return`
- `seq` (bloco sequencial)
- `true`
- `var`
- `while`
- `s_channel` (canal síncrono)
- `c_channel` (canal comum/assíncrono)

Tipos

- `number`
- `string`
- `bool`
- `void`
- `list`
- `dict`

- any

Literais

- NUMBER_LITERAL: sequência de dígitos, opcionalmente com ponto decimal
- STRING_LITERAL: sequência de caracteres entre aspas duplas ("...")
- IDENTIFIER: início com letra ou `_`, seguido de letras, dígitos ou `_`

Operadores

- Aritméticos: `+`, `-`, `*`, `/`, `%`
- Comparação: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Lógicos: `&&`, `||`, `!`
- Atribuição: `=`

Delimitadores

- `(, )` - Parênteses
- `{, }` - Chaves
- `[, ]` - Colchetes
- `,` - Vírgula
- `;` - Ponto-e-vírgula
- `:` - Dois-pontos
- `.` - Ponto
- `->` - Seta

3 Metodologia e Modelagem de Software

3.1 Gerenciamento de Projeto (Backlogs)

3.1.1 Product Backlog

Funcionalidade	Descrição	Prioridade
Análise Léxica (Lexer)	Tokenização e reconhecimento de padrões de entrada	Alta
Análise Sintática (Parser)	Construção da AST através de parsing descendente recursivo	Alta
Análise Semântica	Verificação de tipos, escopo e validação de declarações	Alta
Geração de Código	Conversão para Three-Address Code	Alta

Intermediário (TAC)		
Geração de Código C	Tradução para código C compilável	Alta
Backend GCC	Compilação e linking com GCC	Alta
Geração de Código ARM	Tradução direto para assembly ARMv7	Média
Runtime Executor	Execução direta de programas Minipar	Média
Canais de Comunicação (Sockets)	Implementação de c_channel e s_channel	Média
Web Interface	Interface gráfica com Gradio para compilação	Média
Sistema de Testes	Suite completa de testes unitários e integração	Alta
Documentação Completa	Tutoriais, guias técnicos e exemplos	Média
Otimizações de Código	Eliminação de código morto, etc...	Baixa

3.1.2 Sprint Backlog

Sprint	Período	Tarefa
S1	26 Set-07 Out	Implementar Lexer
S1	26 Set-07 Out	Parser
S1	26 Set-07 Out	Definição de AST Nodes
S1	26 Set-07 Out	Testes Lexer
S1	26 Set-07 Out	Testes Parser
S2	08-14 Out	Symbol Table Hierárquica
S2	08-14 Out	Semantic Analyzer
S2	08-14 Out	Type Checking Completo
S2	08-14 Out	Detecção de Erros

S2	08-14 Out	Testes de Semântica
S3	15-21 Out	TAC Generator
S3	15-21 Out	C Code Generator
S3	15-21 Out	ARM Code Generator
S3	15-21 Out	GCC Backend
S3	15-21 Out	Built-in Functions
S3	15-21 Out	Testes Codegen
S4	22-24 Out	Runtime Executor
S4	22-24 Out	Variable Management
S4	22-24 Out	Function Execution
S4	22-24 Out	Control Flow Runtime
S4	22-24 Out	Client Channel
S4	22-24 Out	Server Channel
S4	22-24 Out	Parallel Execution
S4	22-24 Out	Test Suite
S4	22-24 Out	Example Programs
S5	25-29 Out	Web Interface com Gradio
S5	25-29 Out	Compiler API
S5	25-29 Out	Scripts Utilitários
S5	25-29 Out	Documentation
S5	25-29 Out	Verification Report
S5	25-29 Out	Examples & Comments

3.2 Modelagem UML

Esta seção detalha a modelagem do sistema utilizando UML, conforme exigido.

3.2.1 Diagrama de Casos de Uso

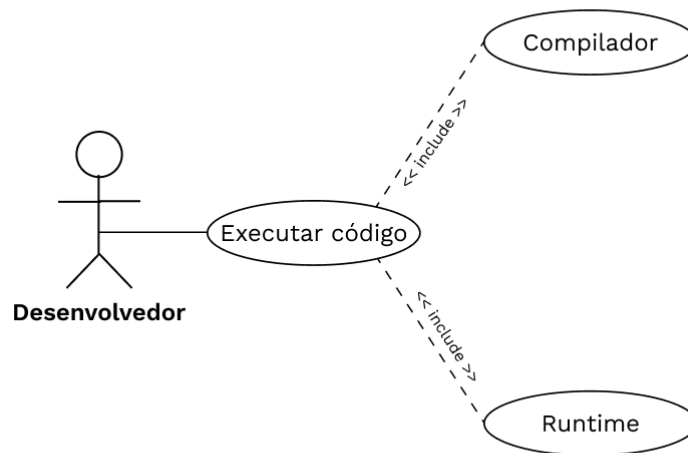


Figura 1: Diagrama de casos de uso Usuário-Minipar

3.2.2 Arquitetura de Componentes

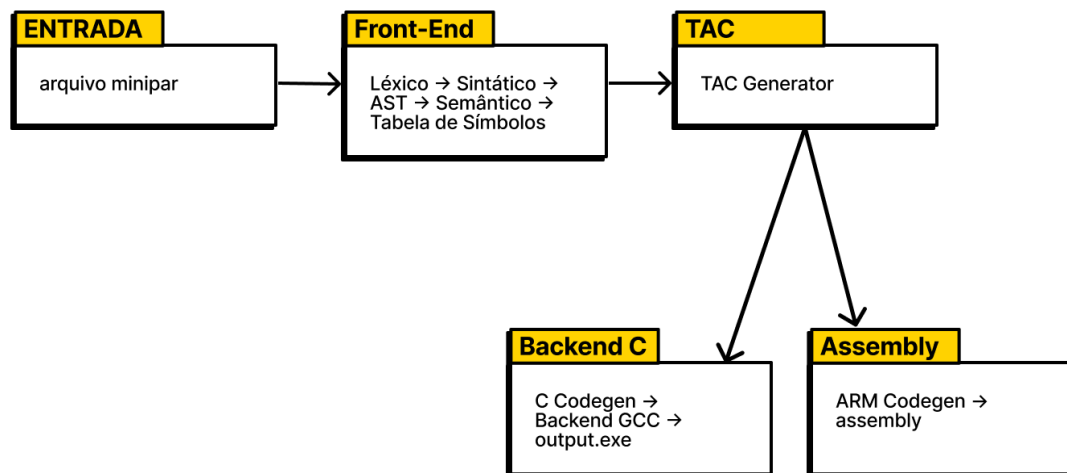


Figura 2: Diagrama de Pacotes da arquitetura

3.2.3 Diagramas de Classe por Componente

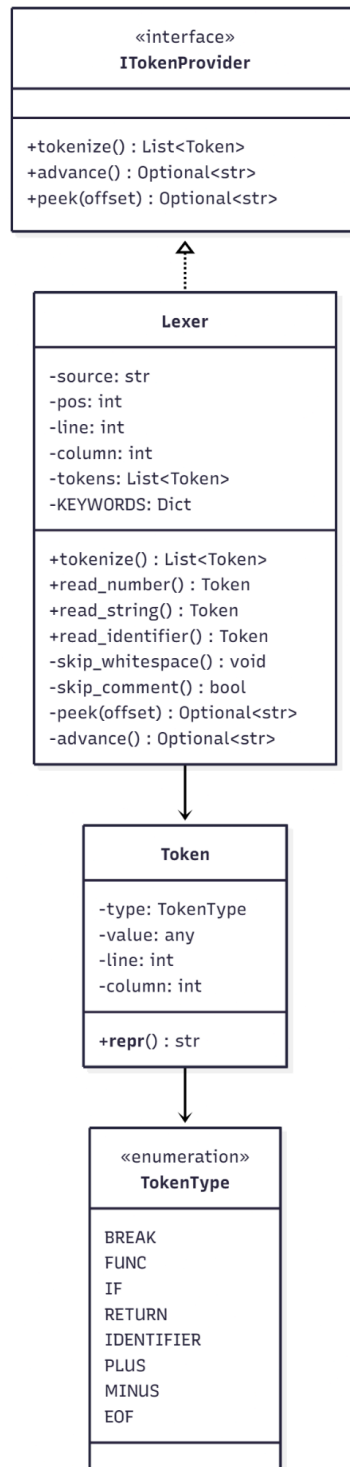


Figura 3: Diagrama de Classes do Componente Léxico

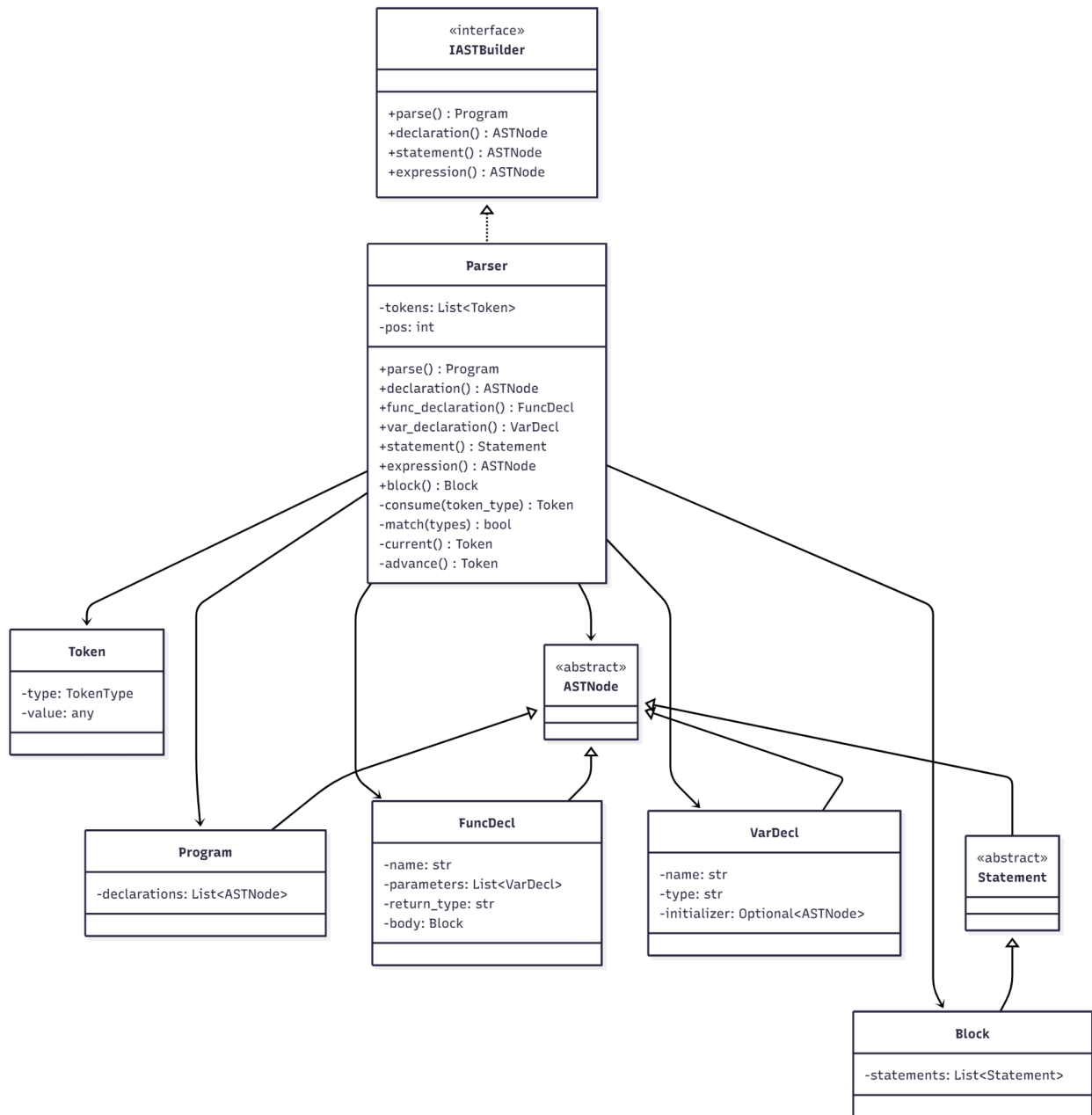


Figura 4: Diagrama de Classes do Componente Sintático

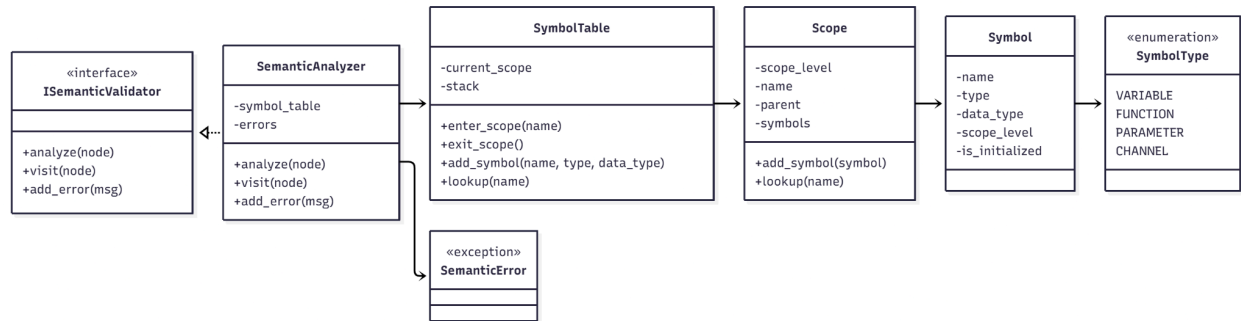


Figura 5: Diagrama de Classes do Componente Semântico

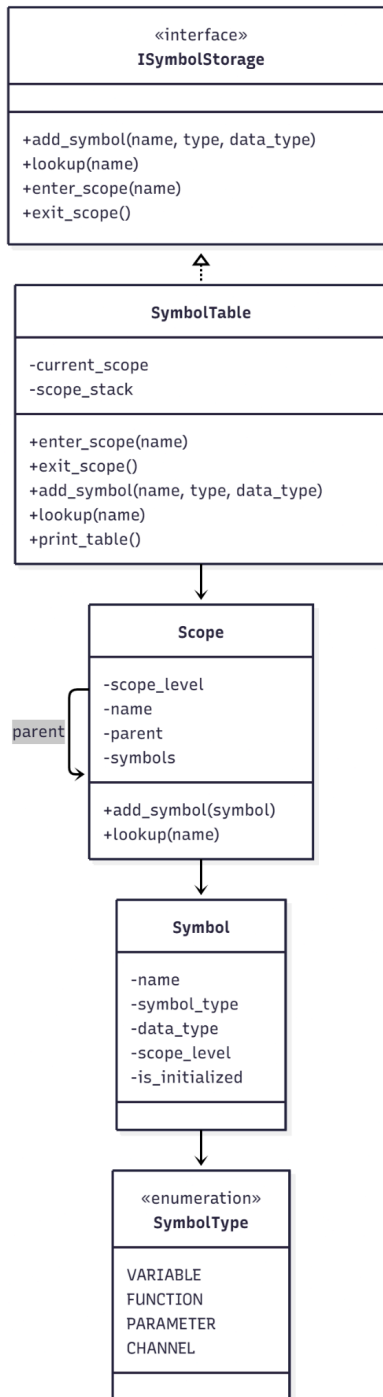


Figura 6: Diagrama de Classes do Componente Tabela de Símbolos

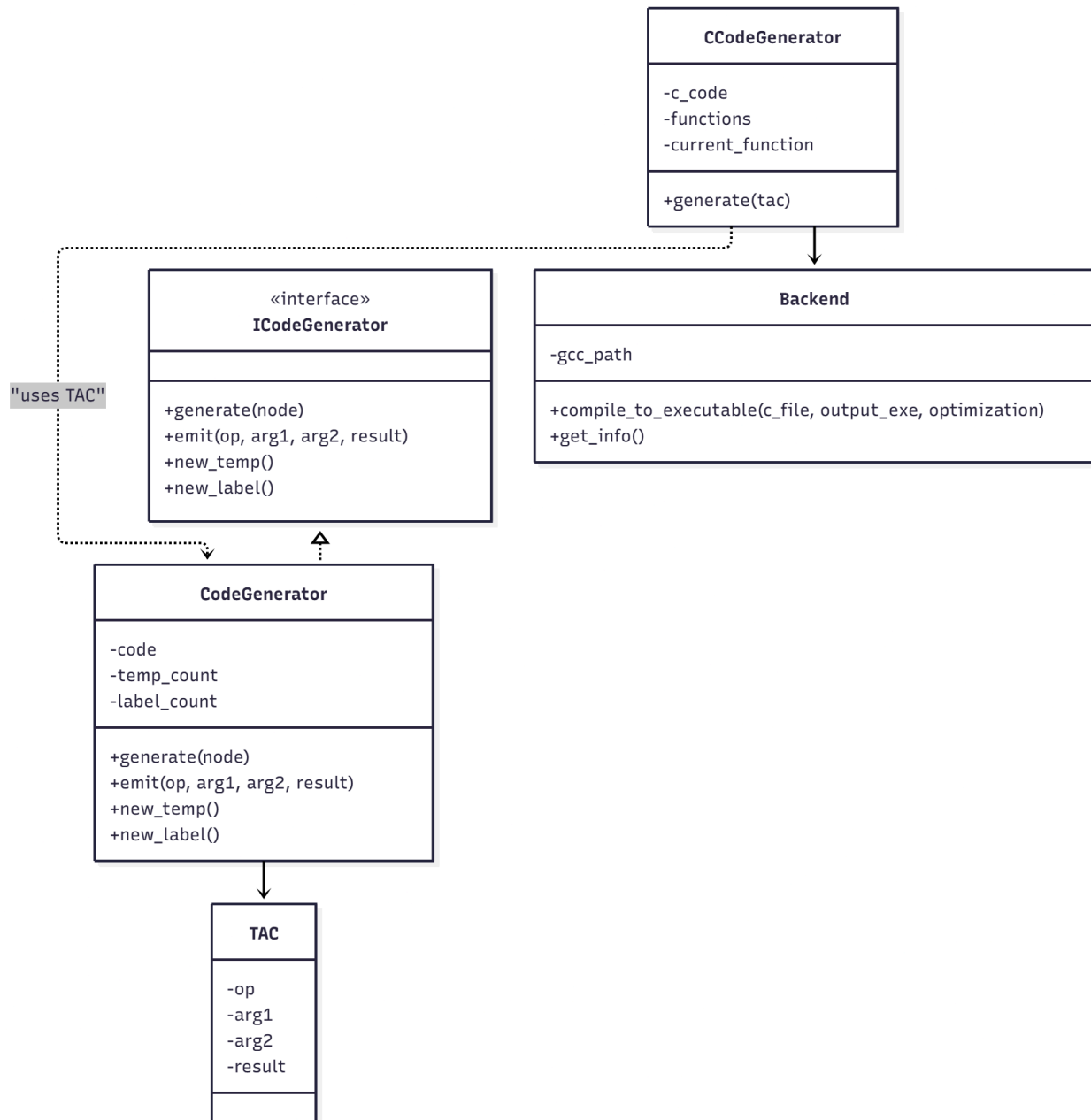


Figura 7: Diagrama de Classes do Componente Gerador de Código

4 Arquitetura e Implementação (Pseudocódigos)

Esta é a seção central do relatório, detalhando o design interno de cada componente do compilador através de pseudocódigos.

4.1 Analisador Léxico (Lexer)

FUNÇÃO Lexer(texto_entrada)

```

pos ← 0
linha ← 1
coluna ← 1
tokens ← []

ENQUANTO pos < comprimento(texto_entrada):
    caractere_atual ← texto_entrada[pos]

    SE caractere_atual é espaço em branco:
        SE caractere_atual = '\n':
            linha ← linha + 1
            coluna ← 1
        SENÃO:
            coluna ← coluna + 1
        pos ← pos + 1

    SE caractere_atual = '/' E próximo_caractere = '/':
        // Comentário de linha única
        ENQUANTO caractere_atual ≠ '\n' E pos < comprimento(texto_entrada):
            pos ← pos + 1

    SE caractere_atual = '/' E próximo_caractere = '*':
        // Comentário multi-linha
        pos ← pos + 2
        ENQUANTO NÃO (caractere_atual = '*' E próximo = '/'):
            SE caractere_atual = '\n':
                linha ← linha + 1
            pos ← pos + 1
        pos ← pos + 2

    SE caractere_atual é dígito:
        valor ← LerNumero(pos, coluna, linha)
        tokens.adicionar(Token(NUMBER_LITERAL, valor, linha, coluna))

    SE caractere_atual = '"':
        valor ← LerString(pos, coluna, linha)
        tokens.adicionar(Token(STRING_LITERAL, valor, linha, coluna))

    SE caractere_atual é letra OU caractere_atual = '_':
        palavra ← LerIdentificadorOuPalavraChave(pos)
        tipo_token ← VerificarPalavraChave(palavra)
        tokens.adicionar(Token(tipo_token, palavra, linha, coluna))

    SE caractere_atual em ['+', '-', '*', '/', '%']:
        tokens.adicionar(Token(OperadorAritmético, caractere_atual, linha, coluna))
        pos ← pos + 1

    SE caractere_atual em ['=', '<', '>', '!']:

```

```
operador ← ProcessarOperadorComparação(pos)
tokens.adicionar(Token(tipo_operador, operador, linha, coluna))
```

SE caractere_atual em ['&', '|']:

```
SE próximo_caractere = caractere_atual:
    operador ← caractere_atual + caractere_atual
    tokens.adicionar(Token(OperadorLógico, operador, linha, coluna))
    pos ← pos + 2
```

SE caractere_atual em ['(', ')', '{', '}', '[', ']', ',', ';', ':', '.']:

```
tokens.adicionar(Token(Delimitador, caractere_atual, linha, coluna))
pos ← pos + 1
```

SE caractere_atual = '-' E próximo = '>':

```
tokens.adicionar(Token(ARROW, '->', linha, coluna))
pos ← pos + 2
```

```
tokens.adicionar(Token(EOF, None, linha, coluna))
```

RETORNA tokens

FIM

FUNÇÃO VerificarPalavraChave(palavra)

```
palavras_chave ← {
    'var': VAR, 'func': FUNC, 'if': IF, 'else': ELSE,
    'while': WHILE, 'break': BREAK, 'continue': CONTINUE,
    'return': RETURN, 'true': TRUE, 'false': FALSE,
    'print': PRINT, 'input': INPUT, 'par': PAR, 'seq': SEQ,
    's_channel': S_CHANNEL, 'c_channel': C_CHANNEL,
    'number': NUMBER, 'string': STRING, 'bool': BOOL, 'void': VOID
}
```

SE palavra em palavras_chave:

```
RETORNA palavras_chave[palavra]
```

SENÃO:

```
RETORNA IDENTIFIER
```

FIM

FUNÇÃO LerNumero(referência pos, coluna, linha)

```
numero_str ← ""
```

ENQUANTO texto[pos] é dígito:

```
numero_str ← numero_str + texto[pos]
pos ← pos + 1
```

// Verificar ponto decimal

SE texto[pos] = '.' E texto[pos+1] é dígito:

```
numero_str ← numero_str + '.'
pos ← pos + 1
```

```

    ENQUANTO texto[pos] é dígito:
        numero_str ← numero_str + texto[pos]
        pos ← pos + 1

    RETORNA CONVERTER_PARA_NUMERO(numero_str)
FIM

FUNÇÃO LerString(referência pos, coluna, linha)
    pos ← pos + 1 // Pular abertura "
    string_valor ← ""

    ENQUANTO pos < comprimento(texto) E texto[pos] ≠ "'":
        SE texto[pos] = '\':
            // Sequência de escape
            pos ← pos + 1
            SE texto[pos] = 'n':
                string_valor ← string_valor + '\n'
            SENÃO SE texto[pos] = 't':
                string_valor ← string_valor + '\t'
            SENÃO SE texto[pos] = "'":
                string_valor ← string_valor + "'"
            SENÃO:
                string_valor ← string_valor + texto[pos]
        SENÃO:
            string_valor ← string_valor + texto[pos]
        pos ← pos + 1

    pos ← pos + 1 // Pular fechamento "
    RETORNA string_valor
FIM

```

4.2 Analisador Sintático (Parser) e AST

4.2.1 AST

```

PROGRAMA      → (DECLARAÇÃO)*
DECLARAÇÃO    → FUNC_DECL | VAR_DECL | CHANNEL_DECL | STATEMENT
FUNC_DECL     → 'func' IDENT '(' PARAMS ')' '->' TIPO '{' BLOCO '}'
VAR_DECL      → 'var' IDENT ':' TIPO ('=' EXPR)?
CHANNEL_DECL  → ('s_channel' | 'c_channel') IDENT CHANNEL_ARGS
STATEMENT     → EXPR_STMT | IF_STMT | WHILE_STMT | RETURN_STMT
               | BREAK | CONTINUE | BLOCO | ATRIB
BLOCO         → '{' (STATEMENT)* '}'
EXPR_STMT     → EXPR ';'
IF_STMT       → 'if' '(' EXPR ')' BLOCO ('else' BLOCO)?
WHILE_STMT    → 'while' '(' EXPR ')' BLOCO
RETURN_STMT   → 'return' EXPR? ';'

```

BREAK → 'break' ';'

 CONTINUE → 'continue' ';'

 PARAMS → (PARAM (',' PARAM)*)?

 PARAM → IDENT ':' TIPO ('=' EXPR)?

 EXPR → EXPR_LÓGICA

 EXPR_LÓGICA → EXPR_COMP (('&&' | '||') EXPR_COMP)*

 EXPR_COMP → EXPR_ARIT (('==' | '!=' | '<' | '>' | '<=' | '>=') EXPR_ARIT)*

 EXPR_ARIT → TERMO (('+' | '-') TERMO)*

 TERMO → FATOR (('*' | '/' | '%') FATOR)*

 FATOR → UNÁRIO

 UNÁRIO → ('!' | '-')? PRIMÁRIA

 PRIMÁRIA → NÚMERO | STRING | IDENT | 'true' | 'false'

 | CHAMADA_FUNC | '(' EXPR ')' | ACESSO_ARRAY

 CHAMADA_FUNC → IDENT '(' ARGS ')'

 ARGS → (EXPR (',' EXPR)*)?

 ACESSO_ARRAY → IDENT '[' EXPR ']'

 TIPO → 'number' | 'string' | 'bool' | 'void' | 's_channel' | 'c_channel'

4.2.2 Analisador Sintático

CLASSE Parser

FUNÇÃO __init__(tokens)

 this.tokens ← tokens

 this.pos ← 0

FUNÇÃO atual()

 SE this.pos < comprimento(this.tokens):

 RETORNA this.tokens[this.pos]

 SENÃO:

 RETORNA this.tokens[comprimento - 1] // EOF

FUNÇÃO avançar()

 token ← this.atual()

 SE this.pos < comprimento(this.tokens) - 1:

 this.pos ← this.pos + 1

 RETORNA token

FUNÇÃO consumir(tipo_esperado, mensagem_erro)

 SE this.atual().tipo ≠ tipo_esperado:

 LANÇAR ERRO(mensagem_erro + " em linha " + this.atual().linha)

 RETORNA this.avançar()

FUNÇÃO casa(tipos_aceitos)

 RETORNA this.atual().tipo EM tipos_aceitos

FUNÇÃO analisar()

 declarações ← []

```

ENQUANTO NÃO this.casa EOF):
    declarações.adicionar(this.declaração())
RETORNA Programa(declarações)

FUNÇÃO declaração()
    SE this.casa(FUNC):
        RETORNA this.decl_função()
    SENÃO SE this.casa(VAR):
        RETORNA this.decl_variável()
    SENÃO SE this.casa(S_CHANNEL, C_CHANNEL):
        RETORNA this.decl_canal()
    SENÃO:
        RETORNA this.comando()

FUNÇÃO decl_função()
    this.consumir(FUNC, "Esperado 'func'")
    nome ← this.consumir(IDENTIFIER, "Esperado nome da função").valor

    this.consumir(LPAREN, "Esperado '('")
    parâmetros ← []
    SE NÃO this.casa(RPAREN):
        parâmetros.adicionar(this.parâmetro())
        ENQUANTO this.casa(COMMA):
            this.avançar()
            parâmetros.adicionar(this.parâmetro())
    this.consumir(RPAREN, "Esperado ')")

    this.consumir(ARROW, "Esperado '->")
    tipo_retorno ← this.tipo_especificador()

    corpo ← this.bloco()
    RETORNA DeclFunção(tipo_retorno, nome, parâmetros, corpo)

FUNÇÃO decl_variável()
    this.consumir(VAR, "Esperado 'var'")
    nome ← this.consumir(IDENTIFIER, "Esperado nome da variável").valor

    this.consumir(COLON, "Esperado ':'")
    tipo ← this.tipo_especificador()

    inicializador ← NULO
    SE this.casa(ASSIGN):
        this.avançar()
        inicializador ← this.expressão()

    this.consumir(SEMICOLON, "Esperado ';'")
    RETORNA DeclVariável(nome, tipo, inicializador)

```

```

FUNÇÃO bloco()
  this.consumir(LBRACE, "Esperado '{'")
  comandos ← []

  ENQUANTO NÃO this.casa(RBRACE, EOF):
    comandos.adicionar(this.comando())

  this.consumir(RBRACE, "Esperado '}")
  RETORNA Bloco(comandos)

FUNÇÃO expressão()
  RETORNA this.expr_lógica()

FUNÇÃO expr_lógica()
  esq ← this.expr_comparação()

  ENQUANTO this.casa(AND, OR):
    op ← this.avançar().valor
    dir ← this.expr_comparação()
    esq ← ExpBinária(esq, op, dir)

  RETORNA esq

FUNÇÃO expr_comparação()
  esq ← this.expr_aritmética()

  ENQUANTO this.casa(EQ, NEQ, LT, GT, LTE, GTE):
    op ← this.avançar().valor
    dir ← this.expr_aritmética()
    esq ← ExpBinária(esq, op, dir)

  RETORNA esq

FUNÇÃO expr_aritmética()
  esq ← this.termo()

  ENQUANTO this.casa(PLUS, MINUS):
    op ← this.avançar().valor
    dir ← this.termo()
    esq ← ExpBinária(esq, op, dir)

  RETORNA esq

FUNÇÃO termo()
  esq ← this.fator()

  ENQUANTO this.casa(MULTIPLY, DIVIDE, MODULO):
    op ← this.avançar().valor

```



```

    dir ← this.fator()
    esq ← ExpBinária(esq, op, dir)

RETORNA esq

FUNÇÃO fator()
    SE this.casa(NOT, MINUS):
        op ← this.avançar().valor
        operando ← this.fator()
        RETORNA ExpUnária(op, operando)

    RETORNA this.primária()

FUNÇÃO primária()
    SE this.casa(NUMBER_LITERAL):
        RETORNA LiteralNumero(this.avançar().valor)

    SE this.casa(STRING_LITERAL):
        RETORNA LiteralString(this.avançar().valor)

    SE this.casa(TRUE, FALSE):
        RETORNA LiteralBool(this.avançar().tipo = TRUE)

    SE this.casa(LPAREN):
        this.avançar()
        expr ← this.expressão()
        this.consumir(RPAREN, "Esperado ')")
        RETORNA expr

    SE this.casa(IDENTIFIER):
        nome ← this.avançar().valor

    SE this.casa(LPAREN):
        // Chamada de função
        this.avançar()
        argumentos ← []
        SE NÃO this.casa(RPAREN):
            argumentos.adicionar(this.expressão())
            ENQUANTO this.casa(COMMA):
                this.avançar()
                argumentos.adicionar(this.expressão())
            this.consumir(RPAREN, "Esperado ')")
        RETORNA ChamadaFunção(nome, argumentos)

    SE this.casa(LBRACKET):
        // Acesso a array
        this.avançar()
        índice ← this.expressão()

```

```

        this.consumir(RBRACKET, "Esperado ']')
        RETORNA AcessoArray(nome, índice)

    RETORNA Identificador(nome)

    LANÇAR ERRO("Expressão esperada em " + this.atual().linha)
FIM CLASSE

```

4.3 Analisador Semântico e Tratamento de Erros

CLASSE AnalisadorSemântico

```

FUNÇÃO __init__()
    this.tabela_símbolos ← TabelaSímbolos()
    this.erros ← []
    this.em_função ← FALSO
    this.em_loop ← FALSO
    this.tipo_retorno_função ← NULO
    this.inicializar_funções_internas()

```

```

FUNÇÃO analisar(nó_ast)
    this.erros ← []
    TENTE:
        this.visitar(nó_ast)
    RETORNA comprimento(this.erros) = 0
    PEGUE Erro_Semântico como e:
        this.erros.adicionar(e.mensagem)
    RETORNA FALSO

```

```

FUNÇÃO visitar(nó)
    nome_método ← "visitar_" + nome_classe(nó)
    método ← obter_atributo(this, nome_método)
    SE método existe:
        RETORNA método(nó)
    SENÃO:

```

LANÇAR Erro_Semântico("Não há visitador para " + nome_classe(nó))

FUNÇÃO visitar_Programa(nó)

PARA CADA declaração EM nó.declarações:

 this.visitar(declaração)

FUNÇÃO visitar_DeclVariável(nó)

 // Verificar se variável já foi declarada no escopo atual

 SE this.tabela_símbolos.buscar_local(nó.nome) ≠ NULO:

 this.adicionar_erro("Variável "" + nó.nome + "" já declarada")

 RETORNA

 // Se há inicializador, verificar tipo

 tipo_inicializador ← NULO

 SE nó.inicializador ≠ NULO:

 tipo_inicializador ← this.visitar(nó.inicializador)

 SE tipo_inicializador ≠ nó.tipo E nó.tipo ≠ "any":

 // Permitir conversão implícita de number para string

 SE NÃO (nó.tipo = "string" E tipo_inicializador = "number"):

 this.adicionar_erro("Tipo incompatível em inicialização: " +
 "esperado " + nó.tipo + ", obtido " + tipo_inicializador)

 // Adicionar símbolo na tabela

 this.tabela_símbolos.adicionar_símbolo(

 nome=nó.nome,

 tipo_símbolo=VARIÁVEL,

 tipo_dados=nó.tipo,

 inicializado=nó.inicializador ≠ NULO

)

```

FUNÇÃO visitar_DeclFunção(nó)
    // Verificar se função já foi declarada
    SE this.tabela_símbolos.buscar_local(nó.nome) ≠ NULO:
        this.adicionar_erro("Função '" + nó.nome + "' já declarada")
    RETORNA

    // Registrar função na tabela de símbolos
    tipos_parâmetros ← [param.tipo PARA param EM nó.parâmetros]
    this.tabela_símbolos.adicionar_símbolo(
        nome=nó.nome,
        tipo_símbolo=FUNÇÃO,
        tipo_dados=nó.tipo_retorno,
        tipos_parâmetros=tipos_parâmetros,
        tipo_retorno=nó.tipo_retorno
    )

    // Entrar em novo escopo
    this.tabela_símbolos.entrar_escopo("func_" + nó.nome)

    // Adicionar parâmetros ao escopo
    PARA CADA param EM nó.parâmetros:
        this.tabela_símbolos.adicionar_símbolo(
            nome=param.nome,
            tipo_símbolo=PARÂMETRO,
            tipo_dados=param.tipo
        )

    // Salvar contexto e analisar corpo
    contexto_anterior_função ← this.em_função

```

```
tipo_anterior ← this.tipo_retorno_função  
this.em_função ← VERDADEIRO  
this.tipo_retorno_função ← nó.tipo_retorno
```

```
this.visitar(nó.corpo)
```

```
// Restaurar contexto  
this.em_função ← contexto_anterior_função  
this.tipo_retorno_função ← tipo_anterior  
this.tabela_símbolos.sair_escopo()
```

FUNÇÃO visitar_Identificador(nó)

```
símbolo ← this.tabela_símbolos.buscar(nó.nome)
```

SE símbolo = NULO:

```
  this.adicionar_erro("Identificador " + nó.nome + " não definido")
```

```
  RETORNA "unknown"
```

```
RETORNA símbolo.tipo_dados
```

FUNÇÃO visitar_ExpBinária(nó)

```
tipo_esq ← this.visitar(nó.esquerda)
```

```
tipo_dir ← this.visitar(nó.direita)
```

// Validar compatibilidade de tipos

ESCOLHA nó.operador:

CASO '+':

SE (tipo_esq = "number" OU tipo_esq = "string") E

(tipo_dir = "number" OU tipo_dir = "string"):

RETORNA tipo_esq

SENÃO:

 this.adicionar_erro("Tipos incompatíveis para +: " +
 tipo_esq + " e " + tipo_dir)

CASO '-', '*', '/', '%':

 SE tipo_esq = "number" E tipo_dir = "number":

 RETORNA "number"

 SENÃO:

 this.adicionar_erro("Operador " + nó.operador +
 " requer number, obtido " + tipo_esq + " e " + tipo_dir)

CASO '==', '!=', '<', '>', '<=', '>=':

 SE this.tipos_compatíveis(tipo_esq, tipo_dir):

 RETORNA "bool"

 SENÃO:

 this.adicionar_erro("Tipos incompatíveis em comparação")

CASO '&&', '||':

 SE tipo_esq = "bool" E tipo_dir = "bool":

 RETORNA "bool"

 SENÃO:

 this.adicionar_erro("Operador lógico requer bool")

RETORNA "unknown"

FUNÇÃO visitar_ChamadaFunção(nó)

 símbolo ← this.tabela_símbolos.buscar(nó.nome)

 SE símbolo = NULO:

 this.adicionar_erro("Função '" + nó.nome + "' não definida")

RETORNA "unknown"

SE símbolo.tipo_símbolo ≠ FUNÇÃO:

 this.adicionar_erro("'' + nó.nome + '' não é uma função")

RETORNA "unknown"

// Validar número de argumentos

SE comprimento(nó.argumentos) ≠ comprimento(símbolo.tipos_parâmetros):

 this.adicionar_erro("Número incorreto de argumentos para '' + nó.nome + ''")

RETORNA símbolo.tipo_retorno

// Validar tipos de argumentos

PARA i ← 0 ATÉ comprimento(nó.argumentos):

 tipo_arg ← this.visitar(nó.argumentos[i])

 tipo_param ← símbolo.tipos_parâmetros[i]

SE NÃO this.tipos_compatíveis(tipo_arg, tipo_param):

 this.adicionar_erro("Tipo de argumento incompatível na posição " + i +
 ": esperado " + tipo_param + ", obtido " + tipo_arg)

RETORNA símbolo.tipo_retorno

FUNÇÃO visitar_ComandoReturn(nó)

SE NÃO this.em_função:

 this.adicionar_erro("'return' fora de função")

RETORNA

SE nó.expressão = NULO:

 // Return sem valor

SE this.tipo_retorno_função ≠ "void":

```
    this.adicionar_erro("Função espera retorno de tipo " +  
        this.tipo_retorno_função)
```

SENÃO:

```
    tipo_expr ← this.visitar(nó.expressão)
```

SE tipo_expr ≠ this.tipo_retorno_função:

```
    this.adicionar_erro("Tipo de retorno incompatível: " +  
        "esperado " + this.tipo_retorno_função +  
        ", obtido " + tipo_expr)
```

FUNÇÃO visitar_ComandoBreak(nó)

SE NÃO this.em_loop:

```
    this.adicionar_erro("'break' fora de loop")
```

FUNÇÃO visitar_ComandoContinue(nó)

SE NÃO this.em_loop:

```
    this.adicionar_erro("'continue' fora de loop")
```

FUNÇÃO visitar_ComandoWhile(nó)

```
    tipo_condição ← this.visitar(nó.condição)
```

SE tipo_condição ≠ "bool":

```
    this.adicionar_erro("Condição do while deve ser bool, obtido " + tipo_condição)
```

```
// Salvar estado de loop
```

```
em_loop_anterior ← this.em_loop
```

```
this.em_loop ← VERDADEIRO
```

```
this.visitar(nó.corpo)
```

```
// Restaurar estado
```



```
this.em_loop ← em_loop_anterior
```

```
FUNÇÃO visitar_Comandolf(nó)
```

```
  tipo_condição ← this.visitar(nó.condição)
```

```
  SE tipo_condição ≠ "bool":
```

```
    this.adicionar_erro("Condição do if deve ser bool, obtido " + tipo_condição)
```

```
  this.visitar(nó.corpo_se)
```

```
  SE nó.corpo_senão ≠ NULO:
```

```
    this.visitar(nó.corpo_senão)
```

```
FUNÇÃO tipos_compatíveis(tipo1, tipo2)
```

```
  // Mesmo tipo
```

```
  SE tipo1 = tipo2:
```

```
    RETORNA VERDADEIRO
```

```
  // any é compatível com tudo
```

```
  SE tipo1 = "any" OU tipo2 = "any":
```

```
    RETORNA VERDADEIRO
```

```
  // Conversões implícitas permitidas
```

```
  SE (tipo1 = "number" E tipo2 = "string") OU
```

```
    (tipo1 = "string" E tipo2 = "number"):
```

```
    RETORNA VERDADEIRO
```

```
  RETORNA FALSO
```

```
FUNÇÃO adicionar_erro(mensagem)
```

```
    this.erros.adicionar(mensagem)
```

FIM CLASSE

4.4 Tabela de Símbolos

CLASSE Símbolo

```
    nome: string
```

```
    tipo_símbolo: enum {VARIÁVEL, FUNÇÃO, PARÂMETRO, CANAL}
```

```
    tipo_dados: string (number, string, bool, void, etc.)
```

```
    escopo_nível: integer
```

```
    linha_declarada: integer
```

```
    inicializado: boolean = true
```

```
    tipos_parâmetros: list[string] // Para funções
```

```
    tipo_retorno: string // Para funções
```

```
    tipo_canal: string // Para canais (c_channel ou s_channel)
```

FIM CLASSE

CLASSE Escopo

```
    nível: integer
```

```
    nome: string
```

```
    escopo_pai: Escopo referência ou NULO
```

```
    símbolos: dicionário {nome: Símbolo}
```

FUNÇÃO adicionar_símbolo(símbolo)

```
    SE símbolo.nome EXISTE em this.símbolos:
```

```
        RETORNA FALSO
```

```
    this.símbolos[símbolo.nome] ← símbolo
```

```
    RETORNA VERDADEIRO
```

FUNÇÃO buscar_local(nome)

```
    SE nome EXISTE em this.símbolos:
```

```

    RETORNA this.símbolos[nome]
RETORNA NULO

FUNÇÃO buscar(nome)
    SE nome EXISTE em this.símbolos:
        RETORNA this.símbolos[nome]
    SE this.escopo_pai ≠ NULO:
        RETORNA this.escopo_pai.buscar(nome)
    RETORNA NULO
FIM CLASSE

CLASSE TabelaSímbolos
    escopo_atual: Escopo
    pilha_escopos: stack[Escopo]
    contador_escopos: integer = 0

FUNÇÃO __init__()
    this.escopo_atual ← novo Escopo(0, "global", NULO)
    this.pilha_escopos ← [this.escopo_atual]

FUNÇÃO entrar_escopo(nome_escopo = "bloco")
    this.contador_escopos ← this.contador_escopos + 1
    novo_escopo ← novo Escopo(this.contador_escopos, nome_escopo, this.escopo_atual)
    this.pilha_escopos.adicionar(novo_escopo)
    this.escopo_atual ← novo_escopo

FUNÇÃO sair_escopo()
    SE comprimento(this.pilha_escopos) > 1:
        this.pilha_escopos.remover_último()
        this.escopo_atual ← this.pilha_escopos[comprimento - 1]

```

FUNÇÃO adicionar_símbolo(nome, tipo_símbolo, tipo_dados, linha, **kwargs)

```
símbolo ← novo Símbolo(
    nome=nome,
    tipo_símbolo=tipo_símbolo,
    tipo_dados=tipo_dados,
    escopo_nível=this.escopo_atual.nível,
    linha_declarada=linha,
    **kwargs
)
RETORNA this.escopo_atual.adicionar_símbolo(símbolo)
```

FUNÇÃO buscar_local(nome)

RETORNA this.escopo_atual.buscar_local(nome)

FUNÇÃO buscar(nome)

RETORNA this.escopo_atual.buscar(nome)

FUNÇÃO obter_nível_escopo()

RETORNA this.escopo_atual.nível

FUNÇÃO obter_todos_símbolos()

RETORNA this.escopo_atual.símbolos.cópia()

FIM CLASSE

4.5 Gerador de Código Intermediário (Três Endereços)

CLASSE GeradorCódigo

FUNÇÃO __init__()

this.código ← []

this.contador_temporário ← 0

```
this.contador_rótulo ← 0  
this.pilha_loops ← [] // [(rótulo_início, rótulo_fim), ...]
```

FUNÇÃO novo_temporário()

```
temp ← "t" + this.contador_temporário  
this.contador_temporário ← this.contador_temporário + 1  
RETORNA temp
```

FUNÇÃO novo_rótulo()

```
rótulo ← "L" + this.contador_rótulo  
this.contador_rótulo ← this.contador_rótulo + 1  
RETORNA rótulo
```

FUNÇÃO emitir(operação, arg1=NULO, arg2=NULO, resultado=NULO)

```
this.código.adicionar(novo InstruçãoTAC(operação, arg1, arg2, resultado))
```

FUNÇÃO gerar(nó_ast)

```
nome_método ← "gen_" + nome_classe(nó_ast)  
método ← obter_atributo(this, nome_método)  
SE método existe:  
    RETORNA método(nó_ast)  
SENÃO:  
    LANÇAR não_implementado(nome_classe(nó_ast))
```

FUNÇÃO gen_Programa(nó)

```
PARA CADA declaração EM nó.declarações:  
    this.gerar(declaração)
```

FUNÇÃO gen_DeclVariável(nó)

```
SE nó.inicializador ≠ NULO:
```

```
valor ← this.gerar(nó.inicializador)
this.emitir('ASSIGN', valor, NULO, nó.nome)
```

FUNÇÃO gen_DeclFunção(nó)

```
this.emitir('FUNC_BEGIN', nó.nome)
```

PARA CADA parâmetro EM nó.parâmetros:

```
this.emitir('PARAM', parâmetro.nome)
```

```
this.gerar(nó.corpo)
```

// Se função não tem return, adicionar um

SE última_instrução ≠ RETURN:

SE nó.tipo_retorno = "void":

```
this.emitir('RETURN', NULO)
```

SENÃO:

```
this.emitir('RETURN', "0")
```

```
this.emitir('FUNC_END', nó.nome)
```

FUNÇÃO gen_Bloco(nó)

PARA CADA comando EM nó.comandos:

```
this.gerar(comando)
```

FUNÇÃO gen_ExpBinária(nó)

```
esq ← this.gerar(nó.esquerda)
```

```
dir ← this.gerar(nó.direita)
```

```
resultado ← this.novo_temporário()
```

```
this.emitir(nó.operador, esq, dir, resultado)
```

RETORNA resultado

FUNÇÃO gen_ExpUnária(nó)

operando $\leftarrow$ this.gerar(nó.operando)

resultado $\leftarrow$ this.novo_temporário()

this.emitir('UNARY', nó.operador, operando, resultado)

RETORNA resultado

FUNÇÃO gen_ChamadaFunção(nó)

PARA CADA argumento EM nó.argumentos:

valor_arg $\leftarrow$ this.gerar(argumento)

this.emitir('PARAM', valor_arg)

resultado $\leftarrow$ this.novo_temporário()

this.emitir('CALL', nó.nome, comprimento(nó.argumentos), resultado)

RETORNA resultado

FUNÇÃO gen_Comandolf(nó)

condição $\leftarrow$ this.gerar(nó.condição)

rótulo_senão $\leftarrow$ this.novo_rótulo()

rótulo_fim $\leftarrow$ this.novo_rótulo()

this.emitir('IF_FALSE', condição, NULO, rótulo_senão)

this.gerar(nó.corpo_se)

this.emitir('GOTO', NULO, NULO, rótulo_fim)

this.emitir('LABEL', rótulo_senão)

SE nó.corpo_senão $\neq$ NULO:

this.gerar(nó.corpo_senão)

```
this.emitir('LABEL', rótulo_fim)
```

```
FUNÇÃO gen_ComandoWhile(nó)
```

```
  rótulo_início ← this.novo_rótulo()
```

```
  rótulo_fim ← this.novo_rótulo()
```

```
  this.pilha_loops.adicionar((rótulo_início, rótulo_fim))
```

```
  this.emitir('LABEL', rótulo_início)
```

```
  condição ← this.gerar(nó.condição)
```

```
  this.emitir('IF_FALSE', condição, NULO, rótulo_fim)
```

```
  this.gerar(nó.corpo)
```

```
  this.emitir('GOTO', NULO, NULO, rótulo_início)
```

```
  this.emitir('LABEL', rótulo_fim)
```

```
  this.pilha_loops.remover_último()
```

```
FUNÇÃO gen_ComandoBreak(nó)
```

```
  (_, rótulo_fim) ← this.pilha_loops[comprimento - 1]
```

```
  this.emitir('GOTO', NULO, NULO, rótulo_fim)
```

```
FUNÇÃO gen_ComandoContinue(nó)
```

```
  (rótulo_início, _) ← this.pilha_loops[comprimento - 1]
```

```
  this.emitir('GOTO', NULO, NULO, rótulo_início)
```

```
FUNÇÃO gen_ComandoReturn(nó)
```


SE nó.expressão ≠ NULO:
 valor ← this.gerar(nó.expressão)

SENÃO:
 valor ← NULO

this.emitir('RETURN', valor)

FUNÇÃO gen_ComandoAtribuição(nó)
 valor ← this.gerar(nó.expressão)
 this.emitir('ASSIGN', valor, NULO, nó.alvo)

FUNÇÃO gen_LiteralNumero(nó)
 RETORNA nó.valor

FUNÇÃO gen_LiteralString(nó)
 RETORNA "" + nó.valor + ""

FUNÇÃO gen_LiteralBool(nó)
 RETORNA nó.valor ? "1" : "0"

FUNÇÃO gen_Identificador(nó)
 RETORNA nó.nome

FIM CLASSE

4.6 Gerador de Código Assembly (ARMv7)

CLASSE GeradorAssemblyARM

FUNÇÃO __init__()
 this.seção_dados ← []
 this.seção_rodata ← []
 this.seção_texto ← []

```
this.variáveis_globais ← conjunto()
```

```
this.funções ← dicionário()
```

```
this.função_atual ← NULO
```

```
this.mapa_registradores ← dicionário()
```

```
this.próximo_registrador ← 4 // r4 em diante
```

```
this.literais_string ← dicionário()
```

```
this.contador_string ← 0
```

```
this.prefixo_rótulo_função ← ""
```

```
this.parâmetros_função ← lista()
```

```
FUNÇÃO gerar(instruções_tac)
```

```
    // Primeira passagem: analisar TAC
```

```
    this._analisar_tac(instruções_tac)
```

```
    // Gerar seções
```

```
    this._gerar_seção_dados()
```

```
    this._gerar_seção_rodata()
```

```
    this._gerar_seção_texto(instruções_tac)
```

```
    // Combinar tudo
```

```
    resultado ← []
```

```
    SE this.seção_dados NÃO está vazia:
```

```
        resultado.estender(this.seção_dados)
```

```
        resultado.adicionar("")
```

```
    SE this.seção_rodata NÃO está vazia:
```

```
resultado.estender(this.seção_rodata)
resultado.adicionar("")
```

```
resultado.estender(this.seção_texto)
resultado.adicionar("")
resultado.adicionar("    .end")
```

```
RETORNA "\n".juntar(resultado)
```

```
FUNÇÃO _analisar_tac(instruções)
    em_função ← FALSO
```

```
PARA CADA instrução EM instruções:
```

```
    SE instrução.operação = 'FUNC_BEGIN':
        em_função ← VERDADEIRO
        this.funções[instrução.arg1] ← 0
    SENÃO SE instrução.operação = 'FUNC_END':
        em_função ← FALSO
```

```
    SENÃO SE NÃO em_função:
```

```
        // Variável global
```

```
        SE instrução.operação = 'ASSIGN':
```

```
            SE NÃO (resultado começa com 't'):
```

```
                this.variáveis_globais.adicionar(instrução.resultado)
```

```
FUNÇÃO _gerar_seção_dados()
```

```
    this.seção_dados.adicionar("    .data")
```

```
PARA CADA var EM this.variáveis_globais:
```

```
    this.seção_dados.adicionar(var + " :")
```

```
    this.seção_dados.adicionar("    .word 0")
```

FUNÇÃO _gerar_seção_rodata()

 this.seção_rodata.adicionar(" .section .rodata")

PARA CADA (string, rótulo) EM this.literais_string.itens():

 this.seção_rodata.adicionar(rótulo + ":",)

 this.seção_rodata.adicionar(" .asciz \"" + string + "\"")

FUNÇÃO _gerar_seção_texto(instruções)

 this.seção_texto.adicionar(" .text")

 this.seção_texto.adicionar(" .global main")

PARA CADA instrução EM instruções:

 this._processar_instrução_tac(instrução)

FUNÇÃO _processar_instrução_tac(instrução)

 ESCOLHA instrução.operação:

 CASO 'FUNC_BEGIN':

 this.função_atual ← instrução.arg1

 this.prefixo_rótulo_função ← instrução.arg1 + " _"

 this.seção_texto.adicionar("")

 this.seção_texto.adicionar(instrução.arg1 + ":",)

 this.seção_texto.adicionar(" push {fp, lr}")

 this.seção_texto.adicionar(" add fp, sp, #4")

 CASO 'FUNC_END':

 this.seção_texto.adicionar(" sub sp, fp, #4")

 this.seção_texto.adicionar(" pop {fp, pc}")

 this.função_atual ← NULO

CASO 'LABEL':

```
rótulo ← this.prefixo_rótulo_função + instrução.arg1  
this.seção_texto.adicionar(rótulo + ":")
```

CASO 'ASSIGN':

```
// Carregar valor em registrador  
reg_destino ← this._obter_registrador(instrução.resultado)  
this._gerar_carga(instrução.arg1, reg_destino)  
this._gerar_armazenamento(reg_destino, instrução.resultado)
```

CASO 'ADD':

```
reg1 ← this._obter_registrador(instrução.arg1)  
reg2 ← this._obter_registrador(instrução.arg2)  
reg_resultado ← this._obter_registrador(instrução.resultado)
```

```
this.seção_texto.adicionar("  add " + reg_resultado + ", " + reg1 + ", " + reg2)
```

CASO 'SUB':

```
reg1 ← this._obter_registrador(instrução.arg1)  
reg2 ← this._obter_registrador(instrução.arg2)  
reg_resultado ← this._obter_registrador(instrução.resultado)
```

```
this.seção_texto.adicionar("  sub " + reg_resultado + ", " + reg1 + ", " + reg2)
```

CASO 'MULT':

```
reg1 ← this._obter_registrador(instrução.arg1)  
reg2 ← this._obter_registrador(instrução.arg2)  
reg_resultado ← this._obter_registrador(instrução.resultado)
```

```
this.seção_texto.adicionar("  mul " + reg_resultado + ", " + reg1 + ", " + reg2)
```

CASO 'DIV':

```
reg1 ← this._obter_registrador(instrução.arg1)
reg2 ← this._obter_registrador(instrução.arg2)
reg_resultado ← this._obter_registrador(instrução.resultado)
```

```
// Usar rotina __aeabi_idiv
this.seção_texto.adicionar("  mov r0, " + reg1)
this.seção_texto.adicionar("  mov r1, " + reg2)
this.seção_texto.adicionar("  bl __aeabi_idiv")
this.seção_texto.adicionar("  mov " + reg_resultado + ", r0")
```

CASO 'IF_FALSE':

```
condição_reg ← this._obter_registrador(instrução.arg1)
rótulo_destino ← this.prefixo_rótulo_função + instrução.resultado

this.seção_texto.adicionar("  cmp " + condição_reg + ", #0")
this.seção_texto.adicionar("  beq " + rótulo_destino)
```

CASO 'GOTO':

```
rótulo_destino ← this.prefixo_rótulo_função + instrução.arg1
this.seção_texto.adicionar("  b " + rótulo_destino)
```

CASO 'PARAM':

```
// Copiar valor para registrador de parâmetro
valor ← instrução.arg1
registrador_param ← "r" + (comprimento(this.parâmetros_função) % 4)

this._gerar_carga(valor, registrador_param)
this.parâmetros_função.adicionar(registrador_param)
```

CASO 'CALL':

```
nome_função ← instrução.arg1  
número_args ← instrução.arg2  
resultado ← instrução.resultado
```

```
this.seção_texto.adicionar("  bl " + nome_função)
```

```
reg_resultado ← this._obter_registrador(resultado)  
this.seção_texto.adicionar("  mov " + reg_resultado + ", r0")
```

```
this.parâmetros_função ← []
```

CASO 'RETURN':

```
SE instrução.arg1 ≠ NULO:  
  reg_valor ← this._obter_registrador(instrução.arg1)  
  this.seção_texto.adicionar("  mov r0, " + reg_valor)
```

FUNÇÃO _obter_registrador(variável)

SE variável EM this.mapa_registradores:

RETORNA this.mapa_registradores[variável]

SE this.próximo_registrador ≤ 11:

```
reg ← "r" + this.próximo_registrador  
this.mapa_registradores[variável] ← reg  
this.próximo_registrador ← this.próximo_registrador + 1  
RETORNA reg
```

SENÃO:

```
// Usar stack  
RETORNA "[fp, #-" + (this.próximo_registrador - 11) * 4 + "]"
```

```

FUNÇÃO _gerar_carga(valor, registrador)
    SE é_número(valor):
        this.seção_texto.adicionar("    movw " + registrador + ", #" + valor)
    SENÃO SE é_string_literal(valor):
        rótulo ← this._registrar_string(valor)
        this.seção_texto.adicionar("    ldr " + registrador + ", =" + rótulo)
    SENÃO:
        // Carregar de memória
        this.seção_texto.adicionar("    ldr " + registrador + ", [fp, #-8]")

FUNÇÃO _gerar_armazenamento(registrador, variável)
    this.seção_texto.adicionar("    str " + registrador + ", [fp, #-8]")

FUNÇÃO _registrar_string(string)
    SE string EM this.literais_string:
        RETORNA this.literais_string[string]

    rótulo ← "str_" + this.contador_string
    this.contador_string ← this.contador_string + 1
    this.literais_string[string] ← rótulo
    RETORNA rótulo

FIM CLASSE

```

5 Tecnologias Utilizadas

Aspecto	Descrição
Versão	2025.1
Linguagem-alvo	MiniPar
Linguagem de Implementação	Python 3.7+
Tipo de Parser	LL(1) Descendente Recursivo

Representação Intermediária	Three-Address Code (TAC)
Arquitetura-alvo Primária	ARMv7
Modo de Execução	Compilação + Interpretação
Suporte a Concorrência	Blocos seq/par + Canais (sockets)

6 Testes e Validação

6.1 Interface do Compilador

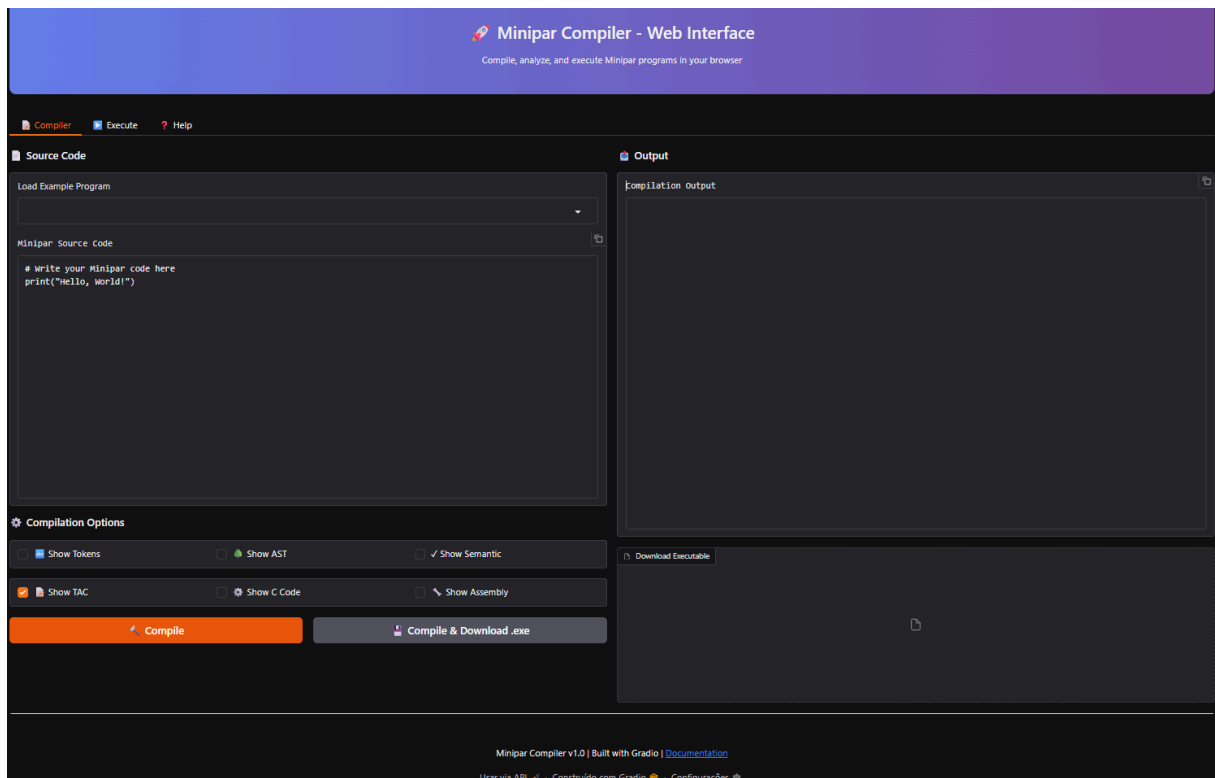


Figura 7: Interface do Compilador Minipar Tela Compilar

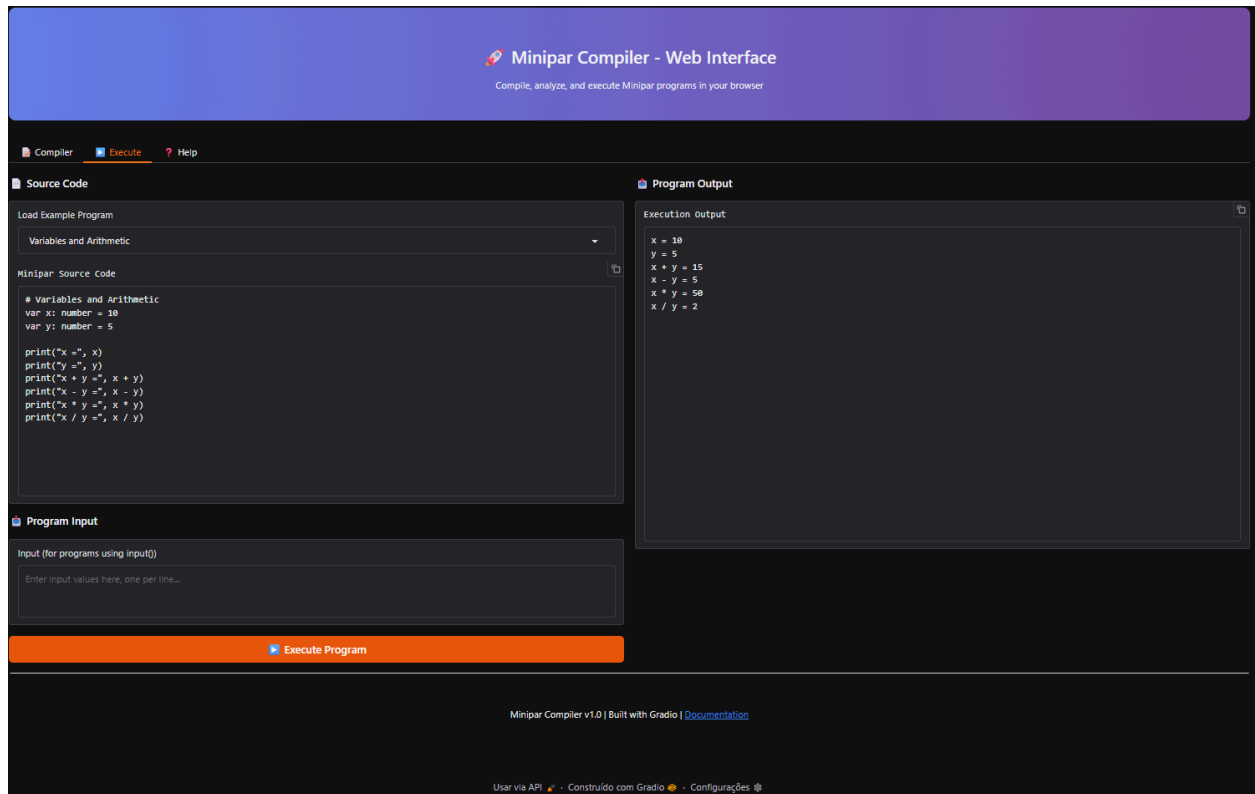


Figura 7: Interface do Compilador Minipar Tela Executar

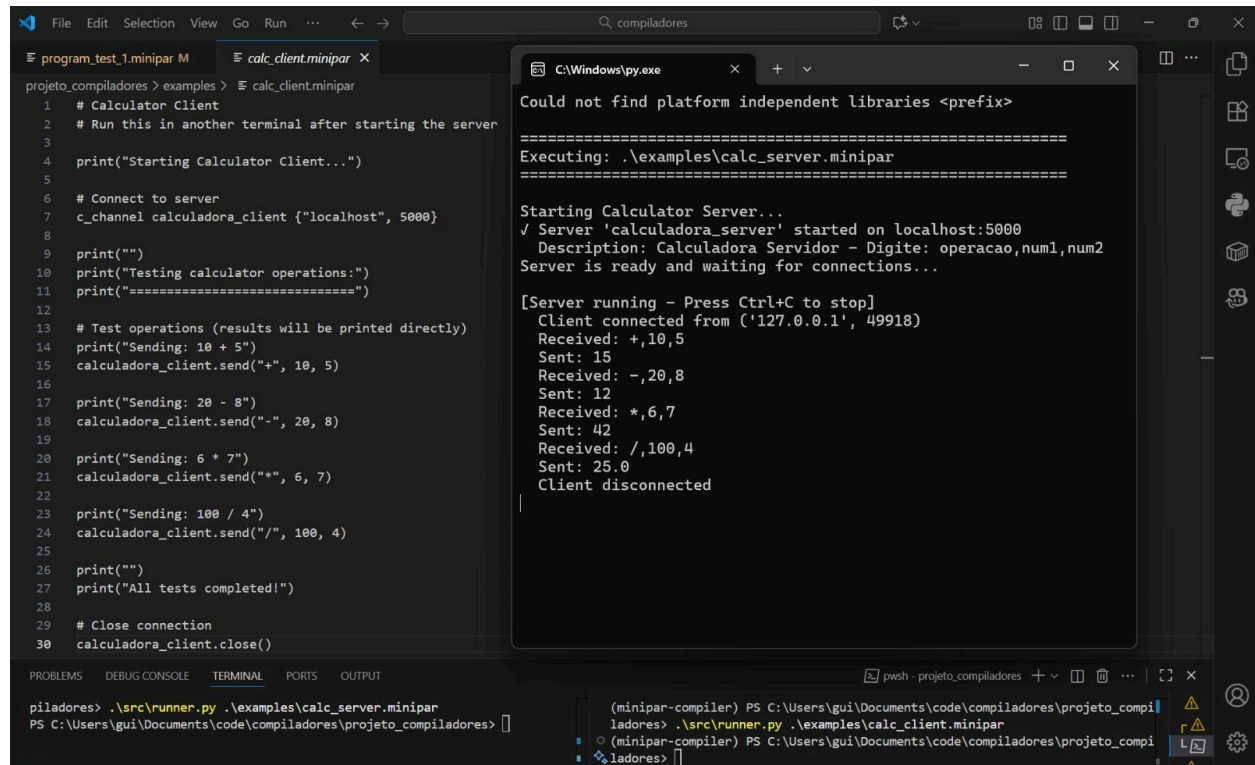
6.2 Testes de Integração



Figura 9: Fluxo de testes

6.3 Execução dos Programas de Teste

6.3.1 Programa de Teste 1: Calculadora Cliente/Servidor



The image shows a Visual Studio Code editor with two files open: `program_test_1.minipar` and `calc_client.minipar`. The `calc_client.minipar` file contains the following Python code:

```
1 # Calculator Client
2 # Run this in another terminal after starting the server
3
4 print("Starting Calculator Client...")
5
6 # Connect to server
7 c_channel calculadora_client {"localhost", 5000}
8
9 print("")
10 print("Testing calculator operations:")
11 print("=====")
12
13 # Test operations (results will be printed directly)
14 print("Sending: 10 + 5")
15 calculadora_client.send("+", 10, 5)
16
17 print("Sending: 20 - 8")
18 calculadora_client.send("-", 20, 8)
19
20 print("Sending: 6 * 7")
21 calculadora_client.send("*", 6, 7)
22
23 print("Sending: 100 / 4")
24 calculadora_client.send("/", 100, 4)
25
26 print("")
27 print("All tests completed!")
28
29 # Close connection
30 calculadora_client.close()
```

The terminal window shows the output of the program execution. It starts with an error message: "Could not find platform independent libraries <prefix>". This is followed by the command `Executing: .\examples\calc_server.minipar`. The output then shows the server starting and the client connecting. The client sends several operations to the server, and the server responds with the results. The client then disconnects.

```
Could not find platform independent libraries <prefix>
=====
Executing: .\examples\calc_server.minipar
=====

Starting Calculator Server...
✓ Server 'calculadora_server' started on localhost:5000
Description: Calculadora Servidor - Digite: operacao,num1,num2
Server is ready and waiting for connections...

[Server running - Press Ctrl+C to stop]
Client connected from ('127.0.0.1', 49918)
Received: +,10,5
Sent: 15
Received: -,20,8
Sent: 12
Received: *,6,7
Sent: 42
Received: /,100,4
Sent: 25.0
Client disconnected
```

Figura 10: Calculadora Executada Terminal Com Conexão Socket Cliente/Servidor

6.3.2 Programa de Teste 2: Fatorial Iterativo (Geração de Assembly ARMv7 e execução no CPUlator).

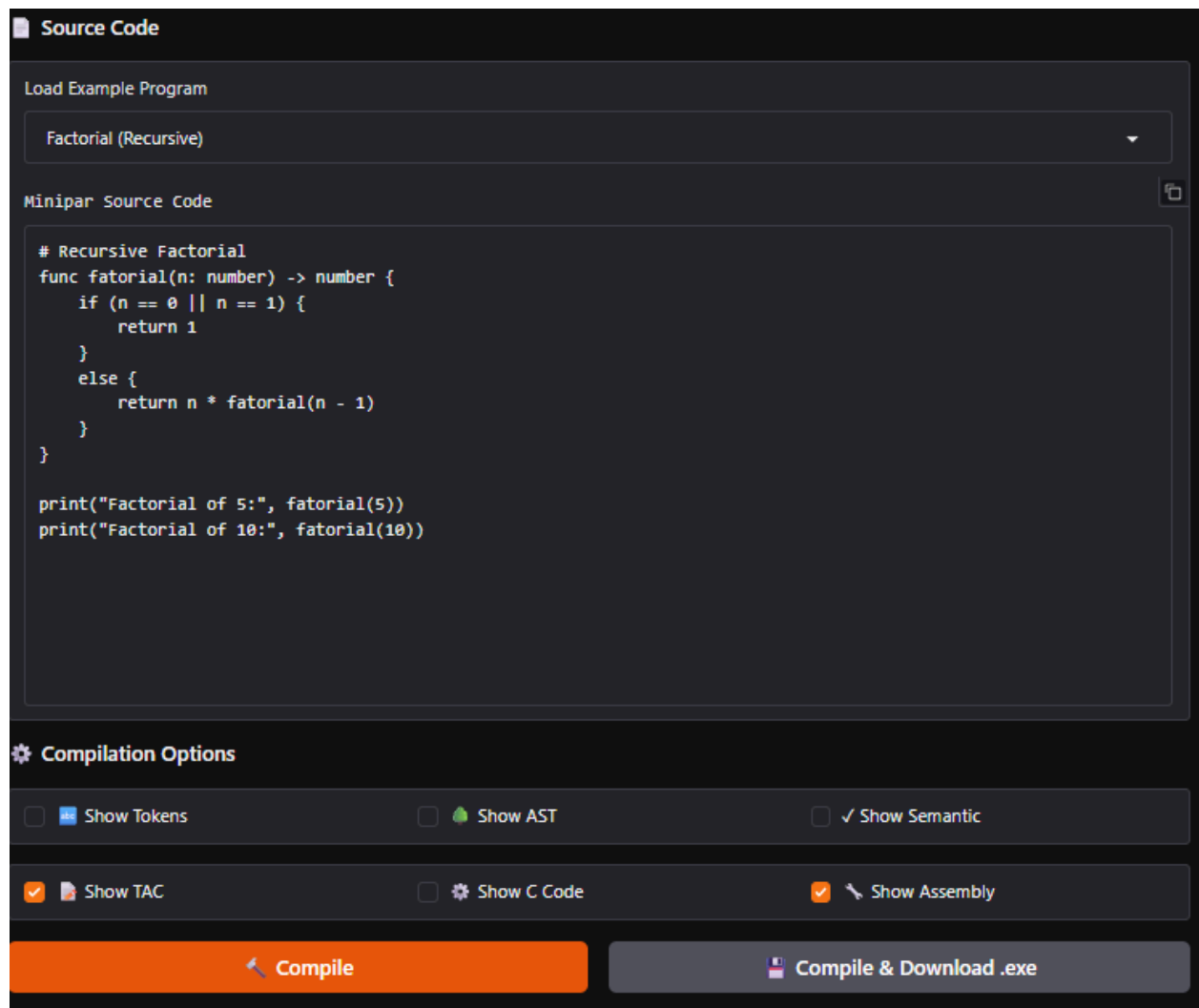
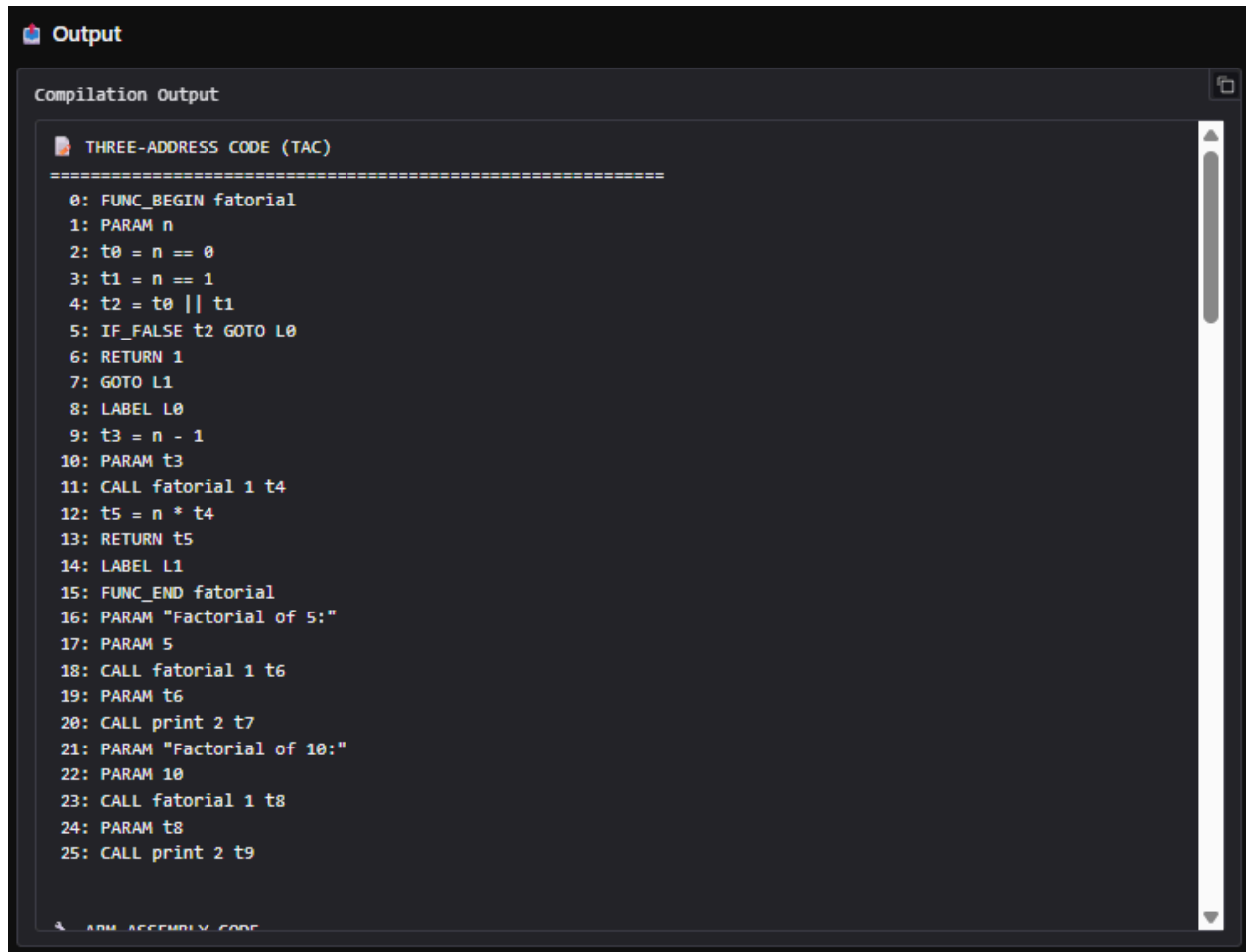


Figura 11: Código Minipar Fatorial



```
Output
Compilation Output
THREE-ADDRESS CODE (TAC)
=====
0: FUNC_BEGIN fatorial
1: PARAM n
2: t0 = n == 0
3: t1 = n == 1
4: t2 = t0 || t1
5: IF_FALSE t2 GOTO L0
6: RETURN 1
7: GOTO L1
8: LABEL L0
9: t3 = n - 1
10: PARAM t3
11: CALL fatorial 1 t4
12: t5 = n * t4
13: RETURN t5
14: LABEL L1
15: FUNC_END fatorial
16: PARAM "Factorial of 5:"
17: PARAM 5
18: CALL fatorial 1 t6
19: PARAM t6
20: CALL print 2 t7
21: PARAM "Factorial of 10:"
22: PARAM 10
23: CALL fatorial 1 t8
24: PARAM t8
25: CALL print 2 t9
ARM ASSEMBLY CODE
```

Figura 12: Saída TAC Fatorial



The screenshot shows a dark-themed IDE window titled "Output". Inside, a tab labeled "Compilation Output" is active. The output text is ARM assembly code, starting with a section separator "===== ARM ASSEMBLY CODE =====". The code defines two string literals: "Factorial of 5:" and "Factorial of 10:". It then sets up the main function, pushes registers, loads the address of the first string, moves the value 5 into register r0, and calls a function named "fatorial".

```
===== ARM ASSEMBLY CODE =====  
  
.data  
@ No global variables  
  
.section .rodata  
.align 2  
.STR0:  
.asciz "Factorial of 5:"  
.STR1:  
.asciz "Factorial of 10:"  
  
.text  
.global main  
.global _start  
.align 2  
  
_start:  
    bl main  
    mov r7, #1      @ exit syscall  
    svc #0          @ make syscall  
  
main:  
    push {r4, r5, r6, r7, lr}  
  
    ldr r0, =.STR0  
    mov r0, #5  
    mov r1, r0  
    bl fatorial  
    mov r4, r0
```

Figura 13: Saída Assembly Fatorial

Refresh

r0	00000000
r1	00000000
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000000
cpsr	000001d3 NZCVI Sv

Registers
Call stack
Trace
Breakpoints
Watchpoints
Symbols
Counters
Settings

Number Display Options
Size: Word
Format: Hexadecimal
Memory words per row: 4

Go to address, label, or register: 00000000 Refresh

Address	Opcode	Disassembly
		12 .global main
		13 .global _start
		14 .align 2
		16 _start:
		17 bl main
		_start:
00000000	eb000001	bl 0xc (0xc: main)
00000004	e3a07001	18 mov r7, #1 @ exit syscall
00000008	ef000000	19 svc #0 @ make syscall
0000000c	e92d40f0	21 main:
00000010	e59f00d0	22 main:
00000014	e3a00005	23 push {r4, r5, r6, r7, lr}
00000018	e1a01000	24 ldr r0, =.STR0
0000001c	eb00000e	25 ldr r0, [pc, #208] ; 0xe8
00000020	e1a04000	26 mov r0, #5 ; 0x5
00000024	e1a05000	27 mov r1, r0
00000028	eb000027	28 bl fatorial
0000002c	e1a05000	29 bl 0x5c (0x5c: fatorial)
		30 mov r4, r0
		31 mov r0, r4
		32 bl print
		33 bl 0xcc (0xcc: print)
		34 mov r5, r0
		35 ldr r6, =.STR1

Editor (Ctrl-E)
Disassembly (Ctrl-D)
Memory (Ctrl-M)

Messages

Compiling...
Code and data loaded from ELF executable into memory. Total size is 280 bytes.
Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2 -o work/asmlZHWU.h.s.o work/asmlZHWU.h.s
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmlZHWU.h.s.elf work/asmlZHWU.h.s.o
Compile succeeded.

Figura 14: Saída Fatorial Assembly Succeeded CpuIator

6.3.3 Programa de Teste 3: Série de Fibonacci (Geração de Assembly ARMv7 e execução no CPUlator).

Source Code

Load Example Program

Factorial (Recursive)

Minipar Source Code

```
func fibonacci(n: number) -> number
{
  if(n == 0){ # Caso base 1
    return 0
  }
  if (n == 1){ # Caso base 2
    return 1
  }
  # Chamada recursiva
  return fibonacci(n - 1) + fibonacci(n - 2) }

var valor: number = 10
print("Fibonacci de ", valor, " é: ", fibonacci(10))
```

Compilation Options

☐ Show Tokens

☐ Show AST

☒ Show Semantic

☒ Show TAC

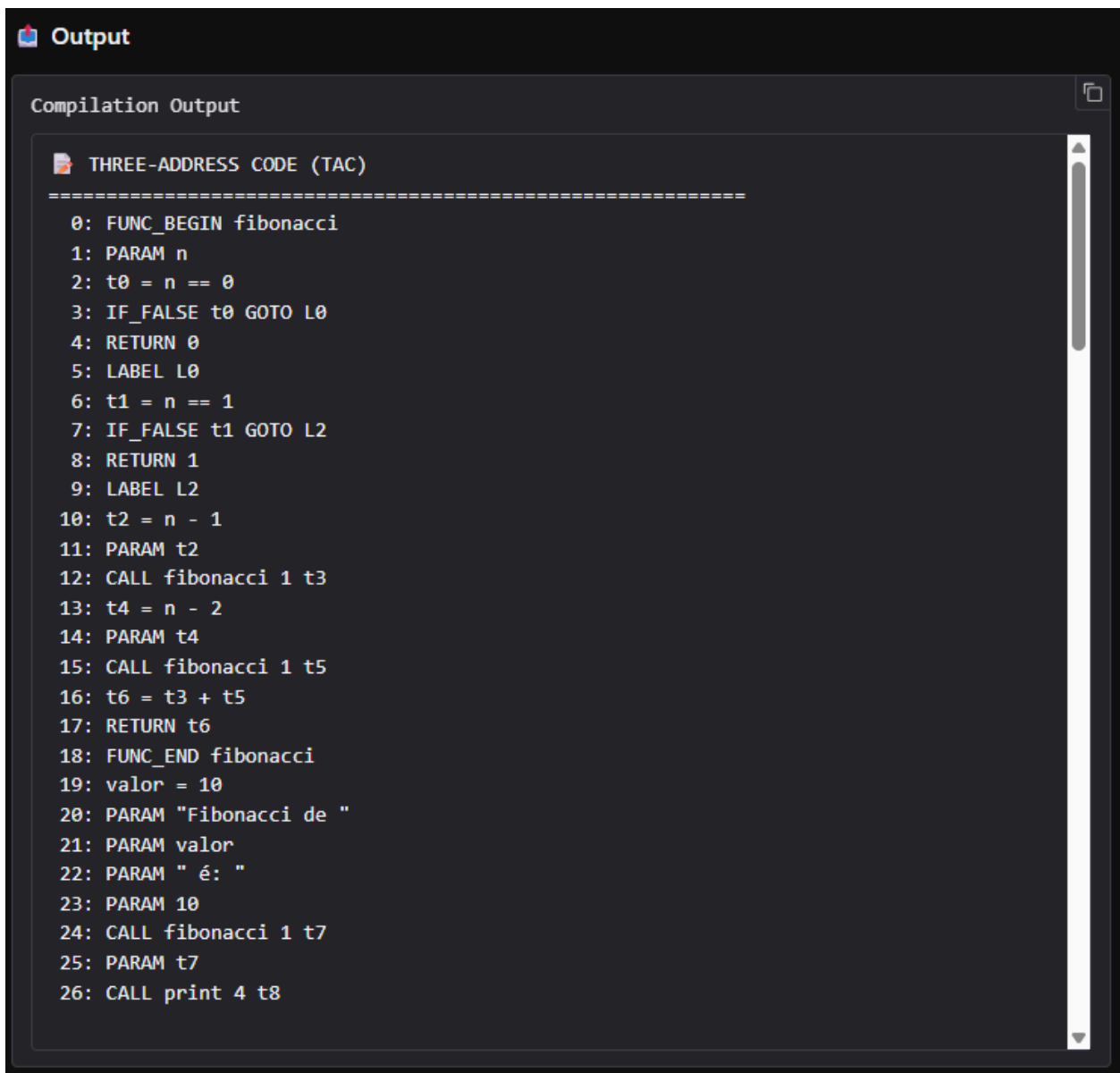
☐ Show C Code

☒ Show Assembly

Compile

Compile & Download .exe

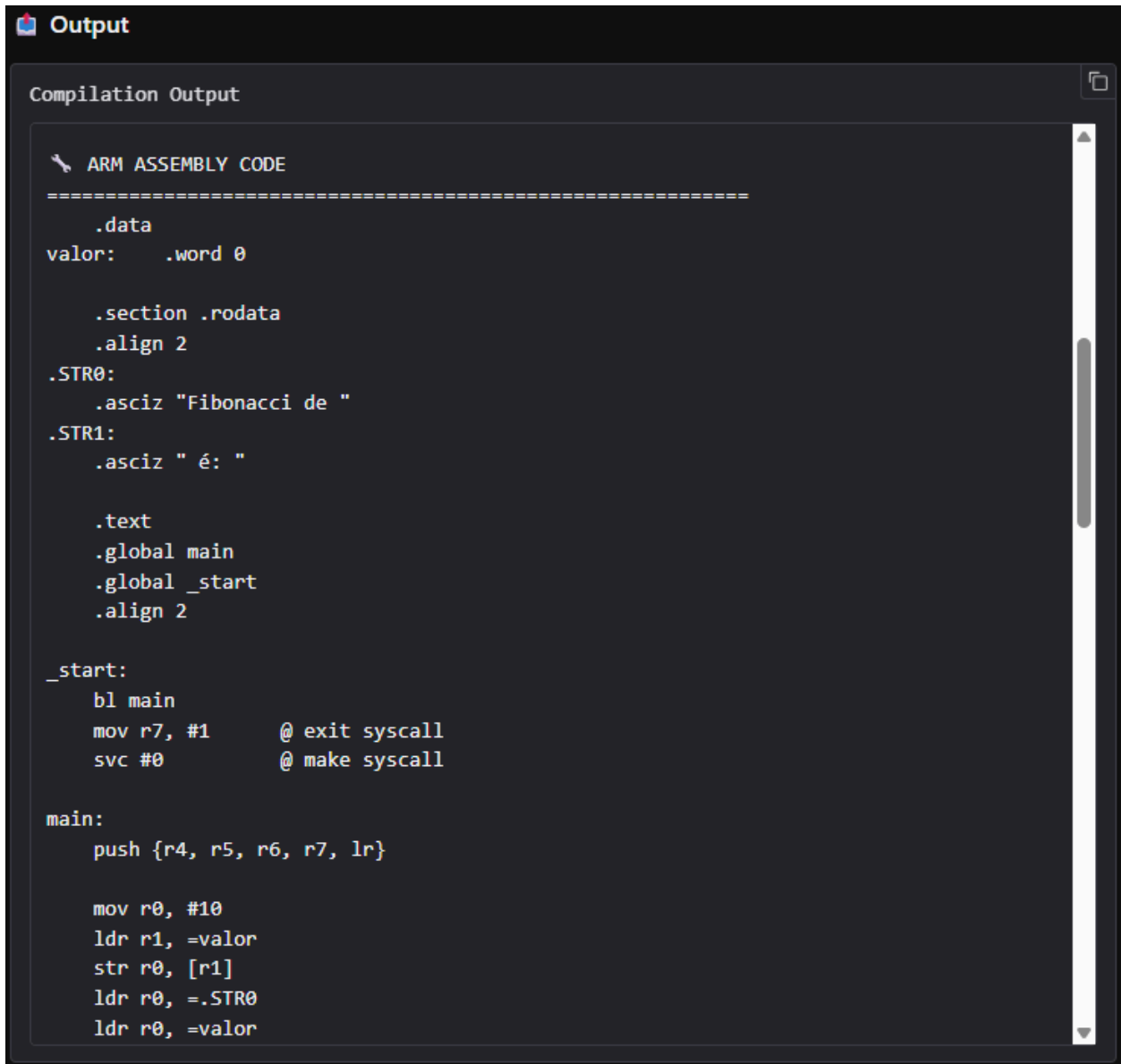
Figura 15: Código Minipar Fibonacci



```
Output
Compilation Output

THREE-ADDRESS CODE (TAC)
=====
0: FUNC_BEGIN fibonacci
1: PARAM n
2: t0 = n == 0
3: IF_FALSE t0 GOTO L0
4: RETURN 0
5: LABEL L0
6: t1 = n == 1
7: IF_FALSE t1 GOTO L2
8: RETURN 1
9: LABEL L2
10: t2 = n - 1
11: PARAM t2
12: CALL fibonacci 1 t3
13: t4 = n - 2
14: PARAM t4
15: CALL fibonacci 1 t5
16: t6 = t3 + t5
17: RETURN t6
18: FUNC_END fibonacci
19: valor = 10
20: PARAM "Fibonacci de "
21: PARAM valor
22: PARAM " é: "
23: PARAM 10
24: CALL fibonacci 1 t7
25: PARAM t7
26: CALL print 4 t8
```

Figura 16: Saída TAC Fibonacci



The screenshot shows a dark-themed IDE window titled "Output". Inside, a tab labeled "Compilation Output" is active. The output displays ARM assembly code for a Fibonacci program. The code is organized into sections: .data, .section .rodata, .text, and labels for _start and main. The .data section defines a variable 'valor' as a word containing 0. The .rodata section defines two strings: "Fibonacci de " and " é: ". The .text section contains the main function logic, including a push of registers, a loop counter initialization, and memory operations to store and retrieve values. The _start label contains instructions to call main, set up exit and syscall registers, and execute the syscall.

```
ARM ASSEMBLY CODE
=====
.data
valor:    .word 0

.section .rodata
.align 2
.STR0:
.asciz "Fibonacci de "
.STR1:
.asciz " é: "

.text
.global main
.global _start
.align 2

_start:
    bl main
    mov r7, #1    @ exit syscall
    svc #0        @ make syscall

main:
    push {r4, r5, r6, r7, lr}

    mov r0, #10
    ldr r1, =valor
    str r0, [r1]
    ldr r0, =.STR0
    ldr r0, =valor
```

Figura 17: Saída Assembly Fibonacci

Refresh

Go to address, label, or register: 00000000 Refresh

Address	Opcode	Disassembly
00000000	eb000001	_start: bl 0xc (0xc: main)
00000004	e3a07001	18 mov r7, #1 @ exit syscall mov r7, #1 ; 0x1
00000008	ef000000	19 svc #0 @ make syscall svc #0 ; 0x0
0000000c	e92d40f0	21 main: push {r4, r5, r6, r7, lr}
00000010	e3a0000a	22 mov r0, #10 ; 0xa
00000014	e59f10e4	23 ldr r1, =valor
00000018	e5810000	24 ldr r1, [pc, #228] ; 0x100 str r0, [r1]
0000001c	e59f00e0	25 ldr r0, =.STR0 ldr r0, [pc, #224] ; 0x104
00000020	e59f00d8	26 ldr r0, =valor
00000024	e5900000	27 ldr r0, [pc, #216] ; 0x100 ldr r0, [r0]
00000028	e59f00d8	28 ldr r0, =.STR1
0000002c	e3a0000a	29 ldr r0, [pc, #216] ; 0x108 mov r0, #10 ; 0xa
00000030	e1a01000	30 mov r1, r0
00000034	e1a02000	31 mov r2, r0
00000038	e1a03000	32 mov r3, r0
0000003c	eb000006	33 bl fibonacci bl 0x5c (0x5c: fibonacci)

Registers Call stack Trace
Breakpoints Watchpoints
Symbols Counters
Settings
Number Display Options
Size: Word
Format: Hexadecimal
Memory words per row: 4
Messages
Compiling...
Code and data loaded from ELF executable into memory. Total size is 296 bytes.
Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2 -o work/asmNZjF0E.s.o work/asmNZjF0E.s
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmNZjF0E.s.elf work/asmNZjF0E.s.o
Compile succeeded.

Figura 18: Saída Fibonacci Assembly Succeeded Cpulator

6.3.4 Programa de Teste 8: Quicksort Iterativo (Geração de Código Intermediário e Assembly ARMv7, com execução no CPUlator).

Source Code

Load Example Program

Factorial (Recursive)

Minipar Source Code

```
# Função para implementar o Quicksort
func quicksort(array: list) -> list {
  if (len(array) <= 1) { # Caso base: vetor com 0 ou 1 elementos já está ordenado
    return array
  }
  else {
    var pivot: any = array[0] # Escolhe o primeiro elemento como pivô

    # Elementos menores ou iguais ao pivô
    var menores: list = []
    for (var x: any in array[1:]) {
      if (x <= pivot) {
        menores.append(x)
      }
    }

    # Elementos maiores ou iguais ao pivô
    var maiores: list = []
    for (var x: any in array[1:]) {
      if (x > pivot) {
        maiores.append(x)
      }
    }

    return quicksort(menores) + [pivot] + quicksort(maiores)
  }
}
```

Compilation Options

☐ Show Tokens

☐ Show AST

☐ ✓ Show Semantic

☒ Show TAC

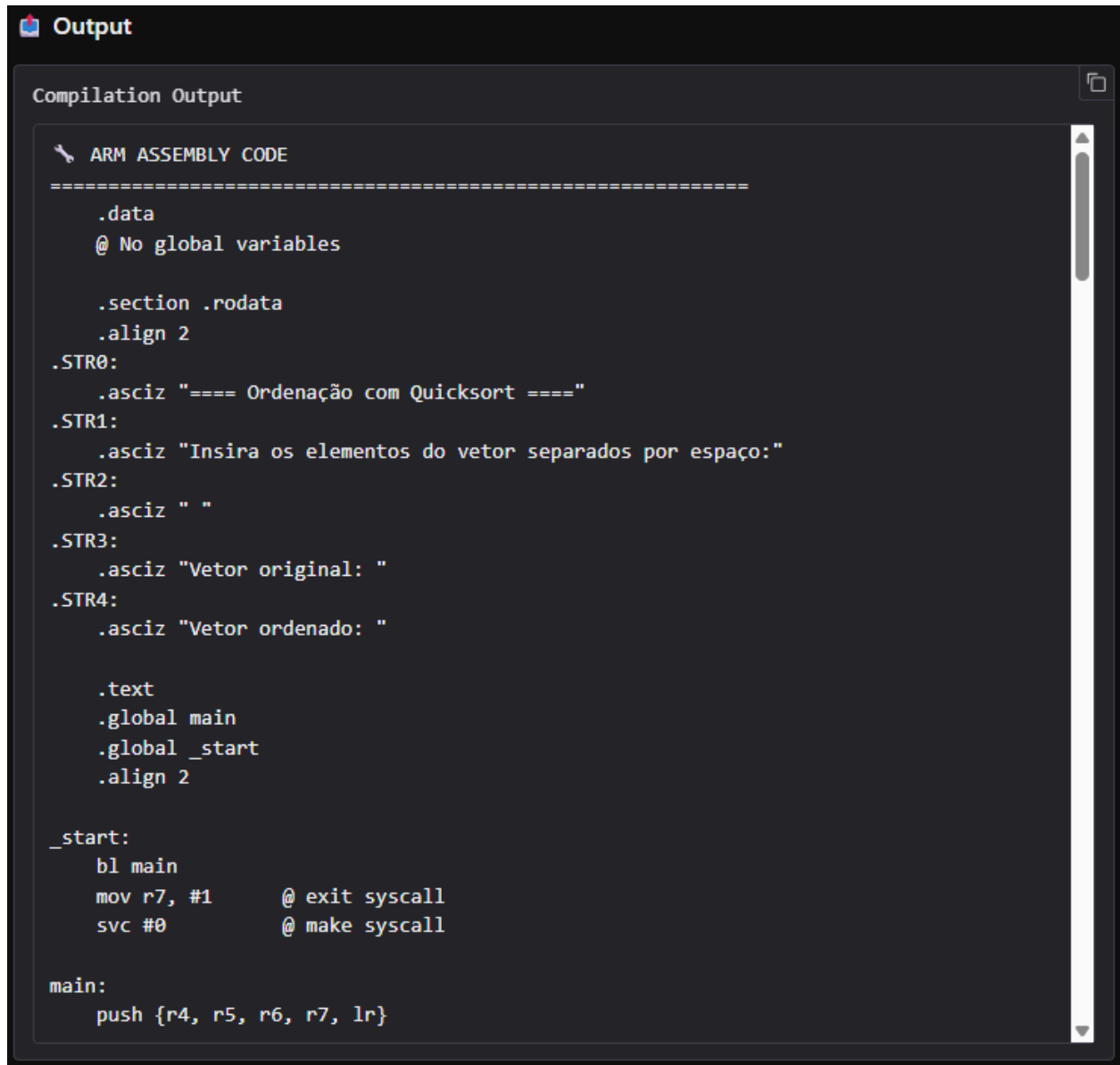
☐ Show C Code

☒ Show Assembly

Compile

Compile & Download .exe

Figura 19: Código Minipar Quicksort



```
ARM ASSEMBLY CODE
=====
.data
@ No global variables

.section .rodata
.align 2
.STR0:
.asciz "==== Ordenação com Quicksort ===="
.STR1:
.asciz "Insira os elementos do vetor separados por espaço:"
.STR2:
.asciz " "
.STR3:
.asciz "Vetor original: "
.STR4:
.asciz "Vetor ordenado: "

.text
.global main
.global _start
.align 2

_start:
    bl main
    mov r7, #1    @ exit syscall
    svc #0        @ make syscall

main:
    push {r4, r5, r6, r7, lr}
```

Figura 20: Saída Assembly QuickSort

7 Conclusão

O desenvolvimento do compilador *Minipar* representou a consolidação prática dos conceitos de construção de compiladores, desde a análise léxica até a geração de código executável. O projeto foi **implementado com sucesso**, abrangendo todas as fases principais: análise léxica, sintática, semântica, geração de código intermediário (TAC), tradução para código C e integração com o backend GCC. A arquitetura modular adotada, aliada à estratégia de testes unitários e de integração, assegurou a consistência entre os componentes e a confiabilidade do processo de compilação como um todo.

Em síntese, o projeto atingiu seus objetivos principais, resultando em um compilador funcional, extensível e bem estruturado, servindo como base sólida para futuras evoluções e estudos sobre técnicas avançadas de compilação e geração de código.

8 Anexos

8.1 Link para o Repositório (GitHub)

https://github.com/HelioJoseRR/projeto_compiladores.git

8.2 Link para o Vídeo Demonstrativo (YouTube)

<https://youtu.be/5bL210jaLDY>

8.3 Link para o Documento (Google Docs)

<https://docs.google.com/document/d/1678HHthb6p5q6JRCT0wfJEpBrOSE45y7Qdu8C1wSSWM/edit?tab=t.0>